

第01章 网关控制器Modbus主机

1.1 练习 01 RT-Thread SRAM驱动

1. 题目要求

由于LWIP协议需要大量内存，我们要提前把系统的内存做统一的管理。

当通过rt_malloc申请内存时，如果默认heap中的申请完了会自动取申请sram2的内存

2. memheap管理算法

在我们使用RTT时，嵌入式的内存资源时比较紧张的，因此我们不希望只使用外部SRAM而放弃内部RAM。因此我们的RTT需要管理多个地址不连续的内存堆。

好在RTT支持memheap管理算法，可以很方便地把多个 memheap（地址可不连续）粘合起来用于系统的 heap 分配。

3. 内存查看

使用命令 free 查看内存

```
msh /> #双击 tab 查看所有命令
RT-Thread shell commands:
clear          - clear the terminal screen
version        - show RT-Thread version information
list_thread    - list thread
list_sem       - list semaphore in system
list_event     - list event in system
list_mutex     - list mutex in system
list_mailbox   - list mail box in system
list_msgqueue  - list message queue in system
list_mempool   - list memory pool in system
list_timer     - list timer in system
list_device    - list device in system
help          - RT-Thread shell help.
ps            - List threads in the system.
free          - Show the memory usage in the system.
ls            - List information about the FILES.
cp            - Copy SOURCE to DEST.
mv            - Rename SOURCE to DEST.
cat           - Concatenate FILE(s)
rm            - Remove(unlink) the FILE(s).
cd            - Change the shell working directory.
pwd           - Print the name of the current working directory.
mkdir         - Create the DIRECTORY.
mkfs          - format disk with file system
mount         - mount <device> <mountpoint> <fstype>
umount        - Unmount device from file system
df            - disk free
echo          - echo string to file
tail          - print the last N - lines data of the given file
fal           - FAL (Flash Abstraction Layer) operate.
sf            - SPI Flash operate.
rt_alarm_dump - dump alarm info
date          - get date and time or set (local timezone) [year month day hou
adc           - adc function
list_fd       - list file descriptor
u_log_flash   - u_log <read|clean> flash log by EasyFlash backend
u_log_savefile - save easyflash logs to file with timestamp
reboot        - Reboot System

msh />free
total   : 54312
used    : 22984
maximum : 23848
msh />
```

• 程序与内存的关系

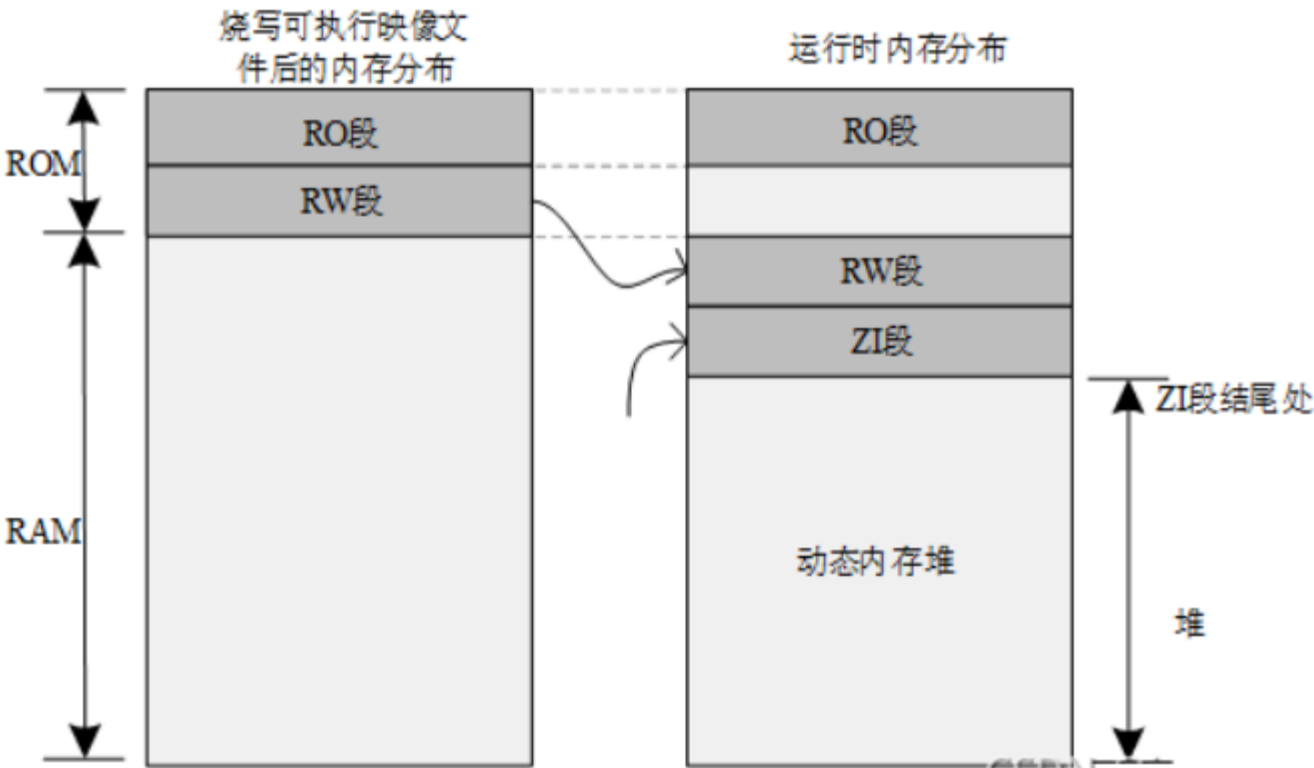
程序运行之前，需要有文件实体被烧录到 STM32 的 Flash 中，是可执行映像文件烧录到 STM32 后的内存分布，

它包含 RO 段和 RW 段两个部分：

- RO 段中保存了 Code、RO-data 的数据，
- RW 段保存了 RW-data 的数据，由于 ZI-data 都是 0，所以未包含在映像文件中。

STM32 启动之后会将 RW 段中的 RW-data（初始化的全局变量）搬运到 RAM 中，但不会搬运 RO 段，执行代码从 Flash 中读取

- ZI：根据编译器给出的 ZI 地址和大小分配出 ZI 段，并将这块 RAM 区域清零。
- 动态内存堆：为未使用的 RAM 空间，RT-Thread 将“ZI 段结尾处”到内存尾部的空间用作内存堆。



可以看到我们的内存一共为54K，这是因为我们程序和系统堆栈也要占用内存，剩余的内存是54K左右，这个和程序有关系。

4. RT-Thread Settings设置

修改内存管理的算法，让所有的内存作为一个对堆对象。

别的功能无需关闭，内核会自动进行合理的调配。

内核 组件 软件包 示例

search

Property	Value
使能 SMP (对称多处理器)	☐
设置数据对齐访问 (Byte)	4
设置线程优先级最大值	32
设置 OS Tick 频率 (Hz)	1000
使用堆栈溢出检查	☒
使能系统钩子功能	☒
使能空闲线程的钩子功能	☒
空闲线程的栈大小	256
使能软件定时器	☒
kservice 服务优化	☒
使能内核调试功能	☒
线程间通信	☒
内存管理	☒
使用内存池	☒
使用最小内存算法	☒
使用slab内存算法	☐
使用内存堆对象	☒
堆内存分配方式	fast mode
系统内存管理方式	Use memheap objects as heap
使能内存跟踪	☐
使用中断服务保护	☐

[rt-thread-kernel-memory-management282]

配置保存后，编译并运行系统，可以看到系统支持命令 list_memheap 了。

```
msh />list_memheap
memheap  pool size  max used size  available size

-----
heap      53984      27844      27076
msh />free
total    : 53984
used     : 26908
maximum  : 27844
msh />
```

5. 增加另外1块内存到系统内

STM32F407 有两块内存

- 0x20000000 开始128K
- 0x10000000 开始64K

drivers\board.h

```
/*----- ROM/RAM CONFIG BEGIN -----*/
```

```
#define ROM_START      ((uint32_t)0x08000000)
#define ROM_SIZE       (1024 * 1024)
#define ROM_END        ((uint32_t)(ROM_START + ROM_SIZE))

#define RAM_START       (0x20000000)
#define RAM_SIZE        (128 * 1024)
#define RAM_END         (RAM_START + RAM_SIZE)

#define RAM1_START      (0x10000000)
#define RAM1_SIZE       (64 * 1024)
#define RAM1_END        (RAM1_START + RAM1_SIZE)

/*----- ROM/RAM CONFIG BEGIN -----*/
```

drivers\board.c

```
int system_sram1_init(void)
{
    static struct rt_memheap memheap_sram1;
    /* Heap initialization */
#ifdef RT_USING_HEAP
    rt_memheap_init(&memheap_sram1, "sram1", (void *)RAM1_START, (rt_size_t)RAM1_SIZE);
#endif
    return 0;
}

RT_WEAK void rt_hw_board_init()
{
    extern void hw_board_init(char *clock_src, int32_t clock_src_freq, int32_t clock_target_freq);

    /* Heap initialization */
#ifdef RT_USING_HEAP
    rt_system_heap_init((void *) HEAP_BEGIN, (void *) HEAP_END);
    system_sram1_init();
#endif

    hw_board_init(BSP_CLOCK_SOURCE, BSP_CLOCK_SOURCE_FREQ_MHZ, BSP_CLOCK_SYSTEM_FREQ_MHZ);

    /* Set the shell console output device */
#ifdef RT_USING_DEVICE && defined(RT_USING_CONSOLE)
    rt_console_set_device(RT_CONSOLE_DEVICE_NAME);
#endif

    /* Board underlying hardware initialization */
#ifdef RT_USING_COMPONENTS_INIT
    rt_components_board_init();
#endif
}
```

编译运行后

```
msh />free
total   : 54088
used    : 26908
maximum : 27844

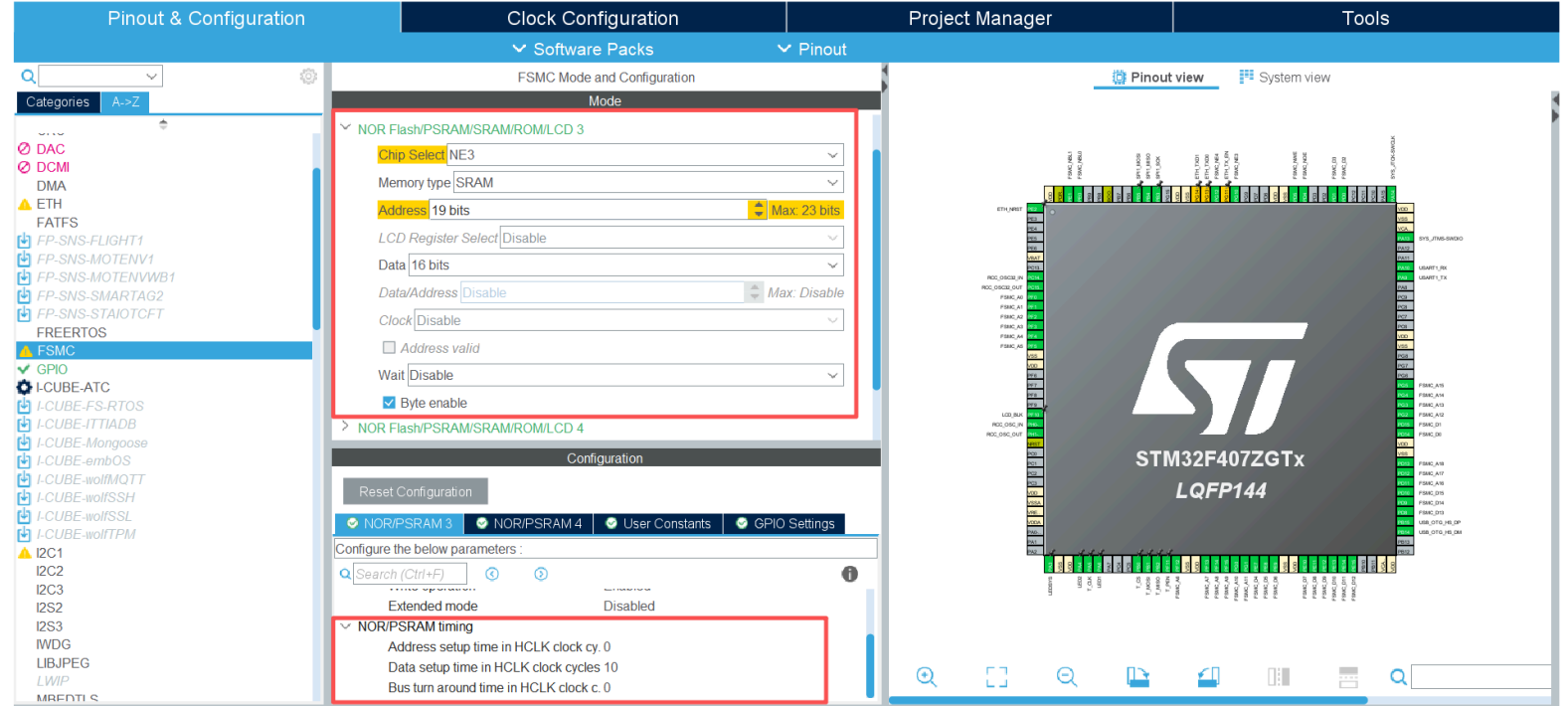
msh />list_memheap
memheap  pool size  max used size available size
-----
sram1    65536      48          65488
heap     54088     27844         27180
msh />
```

内存增加了64K， 对片内的另外一块内存进行了有效的管理。

6. 配置Cubemx的FSMC支持

增加外部的1M 内存。

配置FSMC的Bank3 内存的支持。



根据生成的代码，封装内存初始化函数

drivers\board.h

```
/*----- ROM/RAM CONFIG BEGIN -----*/

#define ROM_START      ((uint32_t)0x08000000)
#define ROM_SIZE       (1024 * 1024)
#define ROM_END        ((uint32_t)(ROM_START + ROM_SIZE))

#define RAM_START       (0x20000000)
#define RAM_SIZE        (128 * 1024)
#define RAM_END         (RAM_START + RAM_SIZE)

#define RAM1_START      (0x10000000)
#define RAM1_SIZE       (64 * 1024)
#define RAM1_END        (RAM1_START + RAM1_SIZE)

#define RAM2_START      (0x68000000)
#define RAM2_SIZE       (1024 * 1024)
#define RAM2_END        (RAM2_START + RAM2_SIZE)

/*----- ROM/RAM CONFIG END -----*/
```

drivers\board.c

```
static void SRAM2_Init(void) ;

int system_sram2_init(void)
{
    static struct rt_memheap memheap_sram2;
    /* Heap initialization */
#ifdef RT_USING_HEAP
    extern void SRAM2_Init(void);
    SRAM2_Init();
    rt_memheap_init(&memheap_sram2, "sram2", (void *)RAM2_START, (rt_size_t)RAM2_SIZE);
#endif
    return 0;
}

/* FSMC initialization function */
static void SRAM2_Init(void)
{
    /* USER CODE BEGIN FSMC_Init 0 */
    extern SRAM_HandleTypeDef hsr3;
    /* USER CODE END FSMC_Init 0 */

    FSMC_NORSRAM_TimingTypeDef Timing = {0};

    /* USER CODE BEGIN FSMC_Init 1 */

    /* USER CODE END FSMC_Init 1 */

    /** Perform the SRAM3 memory initialization sequence
    */
    hsr3.Instance = FSMC_NORSRAM_DEVICE;
    hsr3.Extended = FSMC_NORSRAM_EXTENDED_DEVICE;
    /* hsr3.Init */
    hsr3.Init.NSBank = FSMC_NORSRAM_BANK3;
    hsr3.Init.DataAddressMux = FSMC_DATA_ADDRESS_MUX_DISABLE;
    hsr3.Init.MemoryType = FSMC_MEMORY_TYPE_SRAM;
    hsr3.Init.MemoryDatawidth = FSMC_NORSRAM_MEM_BUS_WIDTH_16;
    hsr3.Init.BurstAccessMode = FSMC_BURST_ACCESS_MODE_DISABLE;
    hsr3.Init.WaitSignalPolarity = FSMC_WAIT_SIGNAL_POLARITY_LOW;
```

```
hsram3.Init.WrapMode = FSMC_WRAP_MODE_DISABLE;
hsram3.Init.WaitSignalActive = FSMC_WAIT_TIMING_BEFORE_WS;
hsram3.Init.WriteOperation = FSMC_WRITE_OPERATION_ENABLE;
hsram3.Init.WaitSignal = FSMC_WAIT_SIGNAL_DISABLE;
hsram3.Init.ExtendedMode = FSMC_EXTENDED_MODE_DISABLE;
hsram3.Init.AsynchronousWait = FSMC_ASYNCHRONOUS_WAIT_DISABLE;
hsram3.Init.WriteBurst = FSMC_WRITE_BURST_DISABLE;
hsram3.Init.PageSize = FSMC_PAGE_SIZE_NONE;
/* Timing */
Timing.AddressSetupTime = 0;
Timing.AddressHoldTime = 15;
Timing.DataSetupTime = 10;
Timing.BusTurnAroundDuration = 0;
Timing.CLKDivision = 16;
Timing.DataLatency = 17;
Timing.AccessMode = FSMC_ACCESS_MODE_A;
/* ExtTiming */

if (HAL_SRAM_Init(&hsram3, &Timing, NULL) != HAL_OK)
{
    Error_Handler( );
}

/* USER CODE BEGIN FSMC_Init 2 */

/* USER CODE END FSMC_Init 2 */
}
```

调用时要注意， 外部内存初始化要放在系统时钟启动后，再调用， 之前初始化会有逻辑问题。

drivers\board.c

```
RT_WEAK void rt_hw_board_init()
{
    extern void hw_board_init(char *clock_src, int32_t clock_src_freq, int32_t clock_target_freq);

    /* Heap initialization */
#ifdef RT_USING_HEAP
    rt_system_heap_init((void *) HEAP_BEGIN, (void *) HEAP_END);
    system_sram1_init();
#endif

    hw_board_init(BSP_CLOCK_SOURCE, BSP_CLOCK_SOURCE_FREQ_MHZ, BSP_CLOCK_SYSTEM_FREQ_MHZ);

    /* Set the shell console output device */
#ifdef RT_USING_DEVICE && defined(RT_USING_CONSOLE)
    rt_console_set_device(RT_CONSOLE_DEVICE_NAME);
#endif

    system_sram2_init(); // 这个函数要放在 系统时钟初始化以后调用， 之前调用会出问题

    /* Board underlying hardware initialization */
#ifdef RT_USING_COMPONENTS_INIT
    rt_components_board_init();
#endif
}
```

7. 运行结果

```
msh />free
total   : 53984
used    : 26908
maximum : 27844
msh />list_memheap
memheap  pool size  max used size available size
-----
sram2    1048576    48          1048528
sram1    65536     48          65488
heap     53984    27844         27076
msh />
```

1.2 练习 02 RT-Thread ETH+LWIP

1. 题目要求

- 移植以太网控制器，并支持LWIP协议。

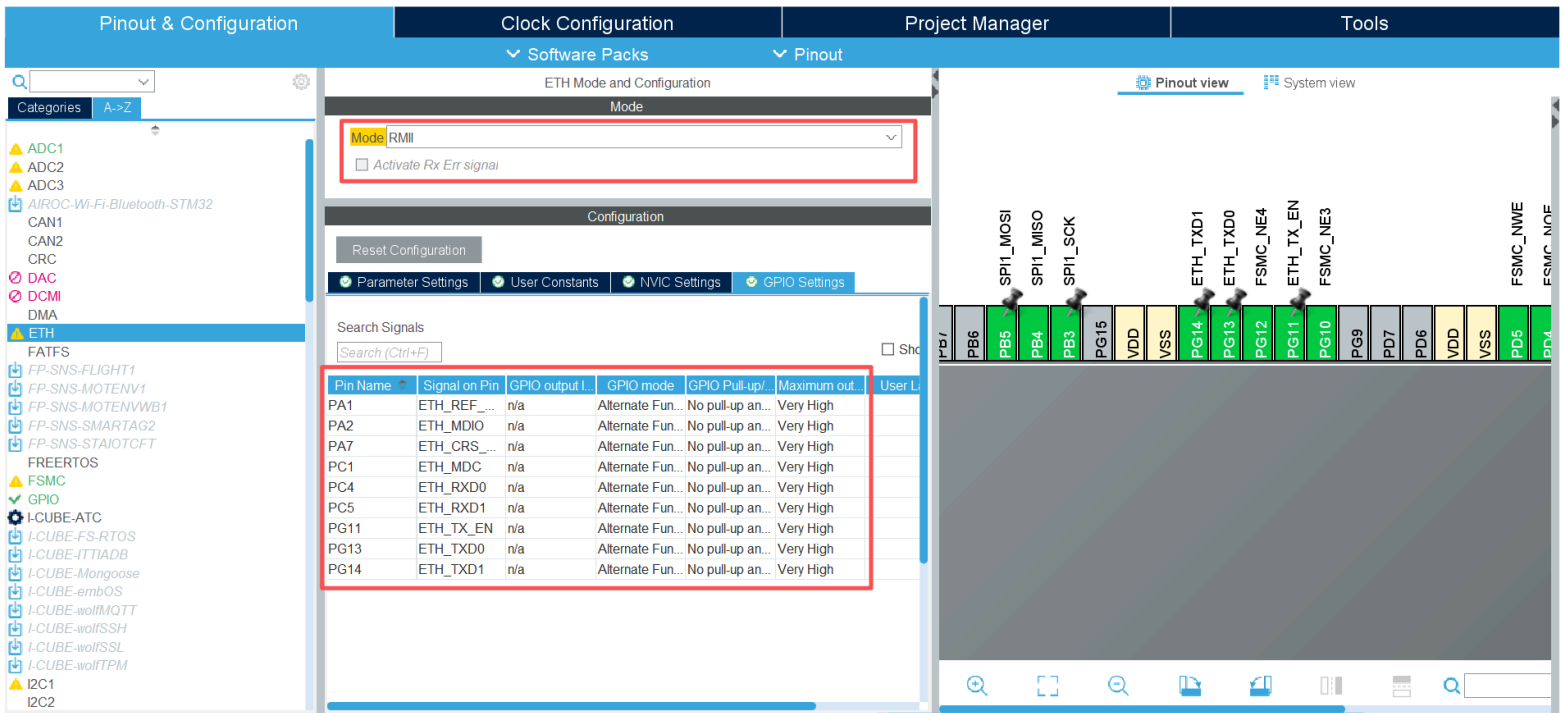
2. RT-Thread Settings设置

点一下就行，不要双击，sal是给ifconfig命令用的，然后ctrl+s保存即可

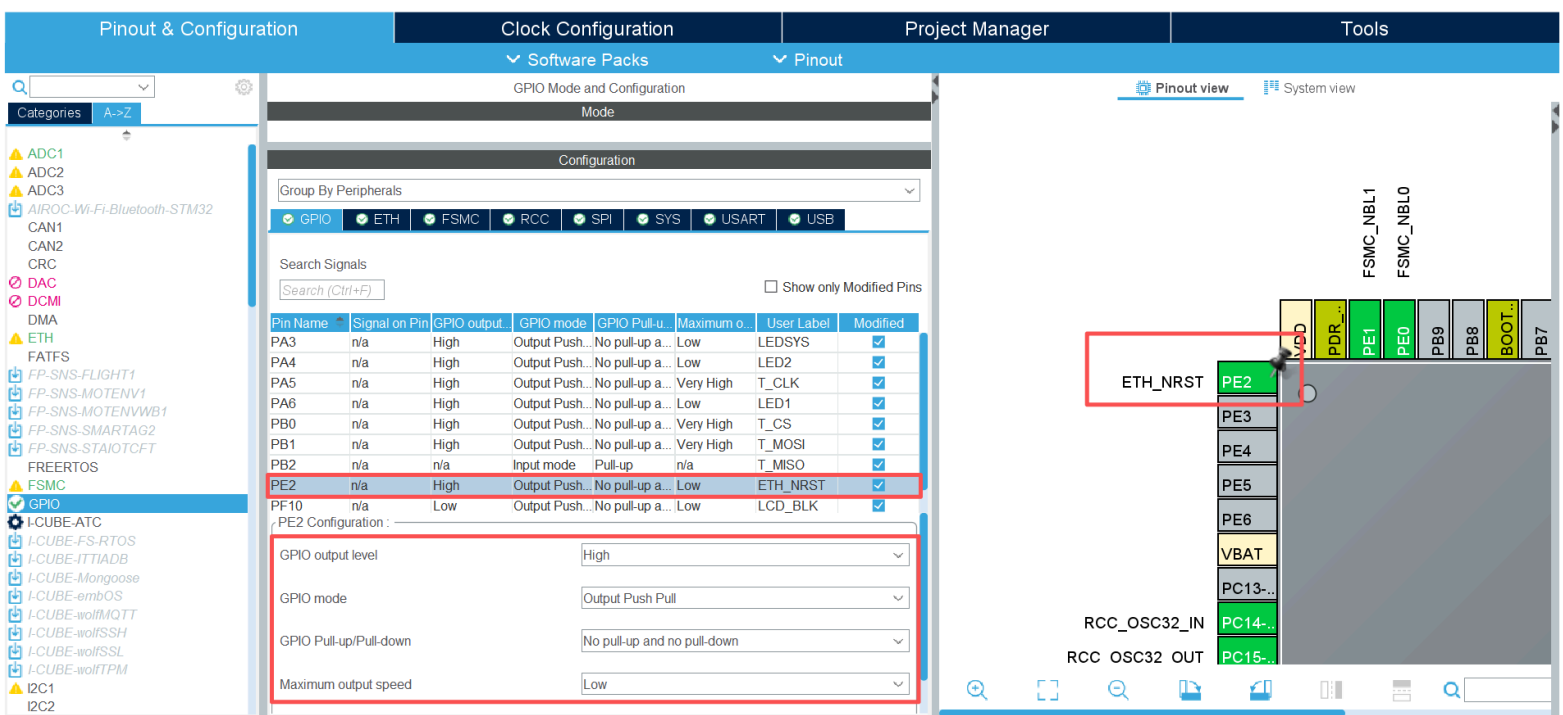


3. 配置Cubemx的引脚

- 配置ETH接口



- 设置网卡的复位引脚



- 添加网卡复位代码

drivers\board.c


```
#define RESET_IO      GET_PIN(E,2)
void phy_reset(void)
{
    rt_pin_mode(RESET_IO, PIN_MODE_OUTPUT);
    rt_pin_write(RESET_IO, PIN_HIGH);
    rt_thread_mdelay(50);
    rt_pin_write(RESET_IO, PIN_LOW);
    rt_thread_mdelay(50);
    rt_pin_write(RESET_IO, PIN_HIGH);
}
```

这段代码是在 drv_eth.c 中调用的。

drivers\drv_eth.c

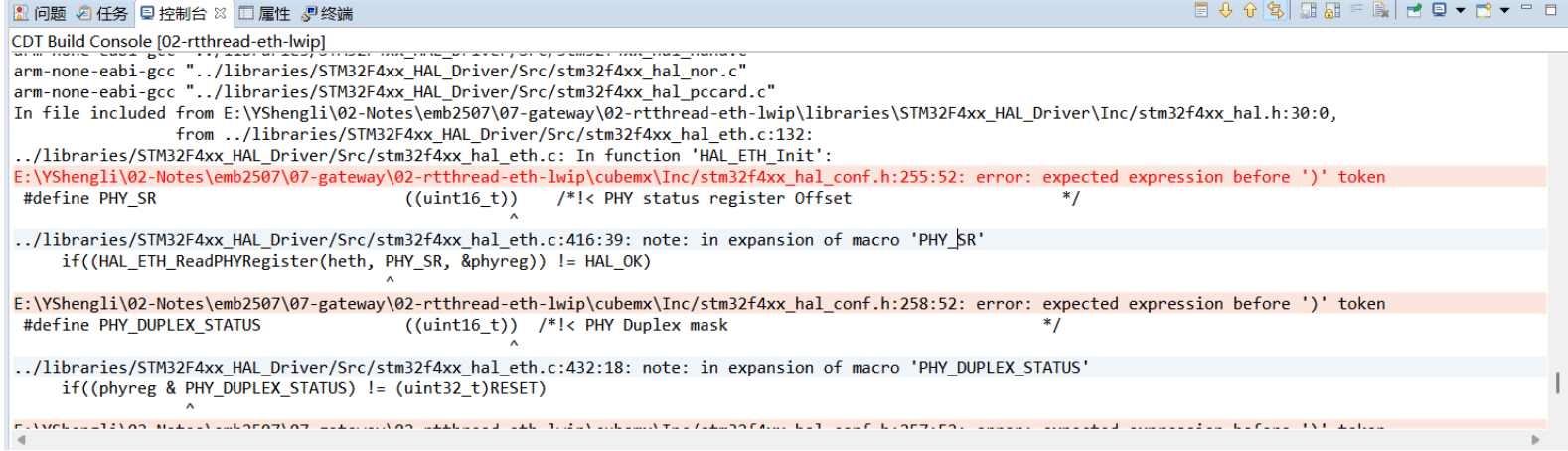
```
extern void phy_reset(void);
/* EMAC initialization function */
static rt_err_t rt_stm32_eth_init(rt_device_t dev)
{
    __HAL_RCC_ETH_CLK_ENABLE();

    phy_reset();

    /* ETHERNET Configuration */
    EthHandle.Instance = ETH;
    EthHandle.Init.MACAddr = (rt_uint8_t *)&stm32_eth_device.dev_addr[0];
    EthHandle.Init.AutoNegotiation = ETH_AUTONEGOTIATION_DISABLE;
    EthHandle.Init.Speed = ETH_SPEED_100M;
    EthHandle.Init.DuplexMode = ETH_MODE_FULLDUPLEX;
    EthHandle.Init.MediaInterface = ETH_MEDIA_INTERFACE_RMII;
    EthHandle.Init.RxMode = ETH_RX_INTERRUPT_MODE;
    ...
}
```

4. 解决编译问题

- 错误1



因为新版本问题，需要手动添加网卡PHY的参数，这个可以参考裸机代码即可。

cubemx\Inc\stm32f4xx_hal_conf.h

```
/* Section 4: Extended PHY Registers */
#define PHY_SR ((uint16_t)) /*!< PHY status register Offset */

#define PHY_SPEED_STATUS ((uint16_t)) /*!< PHY Speed mask */
#define PHY_DUPLEX_STATUS ((uint16_t)) /*!< PHY Duplex mask */

#define PHY_SR ((uint16_t)0x1F) /*!< PHY status register Offset */

#define PHY_SPEED_STATUS ((uint16_t)0x0004) /*!< PHY Speed mask */
#define PHY_DUPLEX_STATUS ((uint16_t)0x0010) /*!< PHY Duplex mask */
```

- 注释代码

cubemx\Src\main.c

```
//ETH_TxPacketConfig TxConfig;
//ETH_DMADescTypeDef DMARxDscrTab[ETH_RX_DESC_CNT]; /* Ethernet Rx DMA Descriptors */
//ETH_DMADescTypeDef DMATxDscrTab[ETH_TX_DESC_CNT]; /* Ethernet Tx DMA Descriptors */

//static void MX_ETH_Init(void);

__WEAK int main(void)
{

    // MX_ETH_Init();

}

//static void MX_ETH_Init(void)
//{
//
//    /* USER CODE BEGIN ETH_Init 0 */
//
//    /* USER CODE END ETH_Init 0 */
//
//    static uint8_t MACAddr[6];
//
//    /* USER CODE BEGIN ETH_Init 1 */
//
//    /* USER CODE END ETH_Init 1 */
//    heth.Instance = ETH;
//    MACAddr[0] = 0x00;
//    MACAddr[1] = 0x80;
//    MACAddr[2] = 0xE1;
//    MACAddr[3] = 0x10;
//    MACAddr[4] = 0x20;
//    MACAddr[5] = 0x30;
//    heth.Init.MACAddr = &MACAddr[0];
//    heth.Init.MediaInterface = HAL_ETH_RMII_MODE;
//    heth.Init.TxDesc = DMATxDscrTab;
//    heth.Init.RxDesc = DMARxDscrTab;
//    heth.Init.RxBuffLen = 1524;
//
//    /* USER CODE BEGIN MACADDRESS */
//
//    /* USER CODE END MACADDRESS */
//
//    if (HAL_ETH_Init(&heth) != HAL_OK)
//    {
//        Error_Handler();
//    }
//
//    memset(&TxConfig, 0 , sizeof(ETH_TxPacketConfig));
//    TxConfig.Attributes = ETH_TX_PACKETS_FEATURES_CSUM | ETH_TX_PACKETS_FEATURES_CRCPAD;
//    TxConfig.ChecksumCtrl = ETH_CHECKSUM_IPHDR_PAYLOAD_INSERT_PHDR_CALC;
//    TxConfig.CRCPadCtrl = ETH_CRC_PAD_INSERT;
//    /* USER CODE BEGIN ETH_Init 2 */
//
//    /* USER CODE END ETH_Init 2 */
//
//}
//}
```

5. 运行结果

```
\ | /
- RT - Thread Operating System
/ | \ 4.1.0 build Nov 4 2025 20:54:15
2006 - 2022 Copyright by RT-Thread team
lwIP-2.0.3 initialized!
RTC hasn't been configured, please use <date> command to config.
adc sample run success! find adc1 device!
[D/FAL] (fal_flash_init:49) Flash device | norflash0 | addr: 0x00
[I/FAL] | name | flash_dev |
offset | length |
[I/FAL] | app | norflash0 |
0x00000000 | 0x00400000 |
[I/FAL] | eflash | norflash0 | 0x00400000 | 0x00a00000 |
[I/FAL] | filesys | norflash0 | 0x00e00000 | 0x00200000 |
[I/FAL] RT-Thread Flash Abstraction
Layer initialize success.
[Flash] EasyFlash v4.1.0 is initialize success.
[Flash] You can get the latest version on https://github.com/armink/EasyFlash .
[I/FAL] The FAL block device (eflash) created successfully
[I/FAL] The FAL block device (filesys) created successfully
msh />10-14 21:04:02 I/sal.skt: Socket Abstraction Layer initialize success.
10-14 21:04:02 I/SFUD: Find a winbond flash chip. Size is 16777216 bytes.
10-14 21:04:02 I/SFUD: norflash0 flash device is initialize success.
10-14 21:04:02 I/SFUD: Probe SPI flash norflash0 by SPI device SPI1 success.
```

10-14 21:04:02 I/SFUD: Create block device on 'eflash' partition successful!
10-14 21:04:02 I/SFUD: Create block device on 'filesys' partition successful!
10-14 21:04:02 I/SFUD: mount 'filesys' partition...
10-14 21:04:02 I/SFUD: 'filesys' partition mounted successfully.
10-14 21:04:02 I/lcd: LCD_Device_Init
10-14 21:04:02 I/lcd: LCD ID:9341

```
msh />ps
thread   pri   status      sp      stack size max used left tick  error
-----
adc1      16  suspend 0x0000015c 0x00000400    35%  0x00000014 000
LEDSYS    24  suspend 0x000000ac 0x00000100    67%  0x00000005 000
tshell    20  running 0x00000274 0x00001000    15%  0x00000006 000
LVGL      20  suspend 0x000000b0 0x00001000    57%  0xfffffbf5 000
usbh       8  suspend 0x000000bc 0x00001000     4%  0x00000014 000
sys work  23  suspend 0x00000084 0x00000800    46%  0x00000007 000
alarmsvc  10  suspend 0x000000b8 0x00000800    08%  0x00000005 000
tcpip     10  suspend 0x000000f8 0x00000400    91%  0x00000013 000
etx       12  suspend 0x000000b8 0x00000400    17%  0x00000010 000
erx       12  suspend 0x000000c8 0x00000400    65%  0x0000000f 000
uilog_asy 30  suspend 0x000000ac 0x00000400    78%  0x00000014 000
tidle0    31  ready   0x0000006c 0x00000100    76%  0x0000001b 000
timer      4  suspend 0x00000080 0x00000200    34%  0x00000009 000
main      10  suspend 0x000000b4 0x00000800    50%  0x0000000c 000
msh />ifconfig
network interface device: e0 (Default)
MTU: 1500
MAC: 00 80 e1 0c 26 56
FLAGS: UP LINK_UP INTERNET_UP DHCP_ENABLE ETHARP BROADCAST IGMP
ip address: 192.168.2.3
gw address: 192.168.2.1
net mask   : 255.255.255.0
dns server #0: 192.168.2.1
dns server #1: 0.0.0.0
msh />ifconfig
network interface device: e0 (Default)
MTU: 1500
MAC: 00 80 e1 0c 26 56
FLAGS: UP LINK_UP INTERNET_UP DHCP_ENABLE ETHARP BROADCAST IGMP
ip address: 192.168.2.3
gw address: 192.168.2.1
net mask   : 255.255.255.0
dns server #0: 192.168.2.1
dns server #1: 0.0.0.0
msh />ping www.baidu.com
60 bytes from 220.181.111.232 icmp_seq=0 ttl=53 time=22 ms
60 bytes from 220.181.111.232 icmp_seq=1 ttl=53 time=22 ms
60 bytes from 220.181.111.232 icmp_seq=2 ttl=53 time=22 ms
60 bytes from 220.181.111.232 icmp_seq=3 ttl=53 time=22 ms
msh />
msh />free
total   : 10820
used    : 10692
maximum : 10820
msh />list_memheap
memheap  pool size  max used size available size
-----
sram2    1048576   31344      1019124
sram1    65536     48         65488
heap     10820    10820      128
msh />
```

现象和裸机测试一样的功能

6. 提升lwip启动速度

cubemx\Inc\stm32f4xx_hal_conf.h

```
/* DP83848_PHY_ADDRESS Address*/
#define DP83848_PHY_ADDRESS
/* PHY Reset delay these values are based on a 1 ms Systick interrupt*/
#define PHY_RESET_DELAY           0x000000FFU
/* PHY Configuration delay */
#define PHY_CONFIG_DELAY          0x000000FFU

#define PHY_READ_TO               0x0000FFFFU
#define PHY_WRITE_TO              0x0000FFFFU
```

修改为：

```
/* PHY Reset delay these values are based on a 1 ms Systick interrupt*/
#define PHY_RESET_DELAY           0x00000000U
/* PHY Configuration delay */
#define PHY_CONFIG_DELAY          0x00000000U
```

PHY_RESET_DELAY ， 网卡复位我们自己实现的， 这里不需要等待。

PHY_CONFIG_DELAY ， 这里验证， 0 也是可以的， 可以明显提速。

1.3 练习 03 RT-Thread FreeModbus

1. 题目要求

- 移植以RS485接口，使能485功能。
- 启用UART3 的485功能。

2. RT-Thread Settings设置

- 选择FreeModbus开源包。

RT-Thread 软件包

modbus

首页 / 6个结果

agile_modbus

轻量级 modbus 协议栈。

loogg

v1.1.2

16501 Apache-2.0

freemodbus

FreeModbus: 开源的 Modbus 主从协议栈

flybreak

v1.6.0

128774 BSD

lhc_modbus

轻量级和高性能的C语言modbus协议栈。

LHC324

v1.0.0

896 Apache-2.0

qmodbus

一款快捷易用的modbus通信协议栈

qiyongzhong0

v1.10

3718 LGPL-2.1

small_modbus

small modbus协议栈

chenbin182

v1.0.0

6157 Apache-2.0

uC_Modbus

uC/Modbus for RT-Thread

Meco Man

latest

1646 Apache-2.0

• 配置FreeModbus

- 使能 RTU 模式
- 使能 RS485例子
- Pin index ： 保持不标， 后续要再修改
- uart number ： 选择 3， 我们使用uart3 作为485通信
- version ： 选择最新

内核 组件 软件包 示例

search

Property	Value
Freemodbus: Modbus 主从堆栈	☒
Master mode	☒
advanced configuration	
Enable RTU master mode	☒
Use Contorl Pin	☒
pin index for RS485 TX/RX select	15
Enable master sample	☒
Test slave device address	1
uart number used by master sample, e.g. 2 means uart2	3
uart baudrate used by master sample	115200
Slave mode	☐
Version	latest
▼	
nanopb: 嵌入式系统的协议缓冲区	☐
Wifi-Host-Driver(WHD) for RT-Thread	☐
ESP-Hosted for RT-Thread	☐
Wi-Fi	
CoAP: 受限应用协议的 C 实现	☐
nopoll: ansi C 中的开源 WebSocket 实现 (RFC 6455)	☐
netutils: RT-Thread 的网络工具集	☐
cmux: connection multiplexing protocol for RT-Thread, support GSM0710_etc	☐
PPP DEVICE: lwIP PPP porting for Cellular Module(2G/3G/4G)	☐
[MODBUS_MASTER_RT_CONTROL_PIN_INDEX]	

3. 添加宏定义

- drivers\board.h

```
/*----- UART CONFIG BEGIN -----*/
```

```

/** After configuring corresponding UART or UART DMA, you can use it.
 *
 * STEP 1, define macro define related to the serial port opening based on the serial port number
 *           such as      #define BSP_USING_UART1
 *
 * STEP 2, according to the corresponding pin of serial port, define the related serial port information macro
 *           such as      #define BSP_UART1_TX_PIN      "PA9"
 *                       #define BSP_UART1_RX_PIN      "PA10"
 *
 * STEP 3, if you want using SERIAL DMA, you must open it in the RT-Thread Settings.
 *           RT-Thread Setting -> Components -> Device Drivers -> Serial Device Drivers -> Enable Serial DMA Mode
 *
 * STEP 4, according to serial port number to define serial port tx/rx DMA function in the board.h file
 *           such as      #define BSP_UART1_RX_USING_DMA
 *
 */

#define BSP_USING_UART1
#define BSP_UART1_TX_PIN      "PA9"
#define BSP_UART1_RX_PIN      "PA10"

#define BSP_USING_UART3
#define BSP_UART3_TX_PIN      "PB10"
#define BSP_UART3_RX_PIN      "PB11"

/*----- UART CONFIG END -----*/
```

修改485控制引脚的参数， 我们使用的是 PG6 。

rtconfig.h

```

/* end of advanced configuration */
#define  PKG_MODBUS_MASTER_RTU
#define  RT_MODBUS_MASTER_USE_CONTROL_PIN
#define  MODBUS_MASTER_RT_CONTROL_PIN_INDEX  15
#define  PKG_MODBUS_MASTER_SAMPLE
```

修改为

```

/* end of advanced configuration */
#define  PKG_MODBUS_MASTER_RTU
#define  RT_MODBUS_MASTER_USE_CONTROL_PIN
#define  MODBUS_MASTER_RT_CONTROL_PIN_INDEX  GET_PIN(G, 6)
#define  PKG_MODBUS_MASTER_SAMPLE
```

4. 源码分析

关键配置（mbconfig.h）：

- 串口参数：波特率115200， 端口3， 偶校验
- 从机数量：16个（MB_MASTER_TOTAL_SLAVE_NUM）
- 支持RTU模式， 不支持ASCII和TCP模式

packages\freemodbus-latest\samples\sample_mb_master.c

```

/*
 * Copyright (c) 2006-2022, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2019-06-21     flybreak     first version
 */

#include <rtthread.h>

#include "mb.h"
#include "mb_m.h"

#ifdef PKG_MODBUS_MASTER_SAMPLE
#define SLAVE_ADDR      MB_SAMPLE_TEST_SLAVE_ADDR
#define PORT_NUM        MB_MASTER_USING_PORT_NUM
#define PORT_BAUDRATE   MB_MASTER_USING_PORT_BAUDRATE
#else
#define SLAVE_ADDR      0x01
#define PORT_NUM        3
#define PORT_BAUDRATE   115200
#endif
#define PORT_PARITY      MB_PAR_EVEN

#define MB_POLL_THREAD_PRIORITY  10
#define MB_SEND_THREAD_PRIORITY  RT_THREAD_PRIORITY_MAX - 1

#define MB_SEND_REG_START  2
#define MB_SEND_REG_NUM    2

#define MB_POLL_CYCLE_MS   500

static void send_thread_entry(void *parameter)
{
    eMBMasterReqErrCode error_code = MB_MRE_NO_ERR;
    rt_uint16_t error_count = 0;
    USHORT data[MB_SEND_REG_NUM] = {0};

    while (1)
    {
        /* Test Modbus Master */
        data[0] = (USHORT)(rt_tick_get() / 10);
        data[1] = (USHORT)(rt_tick_get() % 10);

        error_code = eMBMasterReqwriteMultipleHoldingRegister(SLAVE_ADDR,          /* salve address */
                                                                MB_SEND_REG_START,      /* register start address */
                                                                MB_SEND_REG_NUM,        /* register total number */
                                                                data,                   /* data to be written */
                                                                RT_WAITING_FOREVER);   /* timeout */

        /* Record the number of errors */
        if (error_code != MB_MRE_NO_ERR)
        {
            error_count++;
        }
    }
}

static void mb_master_poll(void *parameter)
{
    eMBMasterInit(MB_RTU, PORT_NUM, PORT_BAUDRATE, PORT_PARITY);
    eMBMasterEnable();

    while (1)
    {
        eMBMasterPoll();
        rt_thread_mdelay(MB_POLL_CYCLE_MS);
    }
}

static int mb_master_sample(int argc, char **argv)
{
    static rt_uint8_t is_init = 0;
    rt_thread_t tid1 = RT_NULL, tid2 = RT_NULL;

    if (is_init > 0)
    {
        rt_kprintf("sample is running\n");
        return -RT_ERROR;
    }
    tid1 = rt_thread_create("md_m_poll", mb_master_poll, RT_NULL, 512, MB_POLL_THREAD_PRIORITY, 10);
    if (tid1 != RT_NULL)
    {
        rt_thread_startup(tid1);
    }
    else
    {
        goto __exit;
    }

    tid2 = rt_thread_create("md_m_send", send_thread_entry, RT_NULL, 512, MB_SEND_THREAD_PRIORITY - 2, 10);
    if (tid2 != RT_NULL)
    {
        rt_thread_startup(tid2);
    }
    else
    {
        goto __exit;
    }
}
```



```
    }

    is_init = 1;
    return RT_EOK;

__exit:
    if (tid1)
        rt_thread_delete(tid1);
    if (tid2)
        rt_thread_delete(tid2);

    return -RT_ERROR;
}
MSH_CMD_EXPORT(mb_master_sample, run a modbus master sample);
```

- 定义保持寄存器二维数组的大小

packages\freemodbus-latest\port\user_mb_app_m.c

```
/*! \brief The total slaves in Modbus Master system. Default 16.
 * \note : The slave ID must be continuous from 1.*/
#define MB_MASTER_TOTAL_SLAVE_NUM          ( 16 )

//Master mode:HoldingRegister variables
USHORT   usMRegHoldStart                   = M_REG_HOLDING_START;
USHORT   usMRegHoldBuf[MB_MASTER_TOTAL_SLAVE_NUM][M_REG_HOLDING_NREGS];
```

调整源码如下:

```
static void send_thread_entry(void *parameter)
{
    EMBMasterReqErrCode error_code = MB_MRE_NO_ERR;
    rt_uint16_t error_count = 0;
    USHORT data[MB_SEND_REG_NUM] = {0};

    while (1)
    {
        /* Test Modbus Master */
        data[0] = (USHORT)(rt_tick_get() / 10);
        data[1] = (USHORT)(rt_tick_get() % 10);

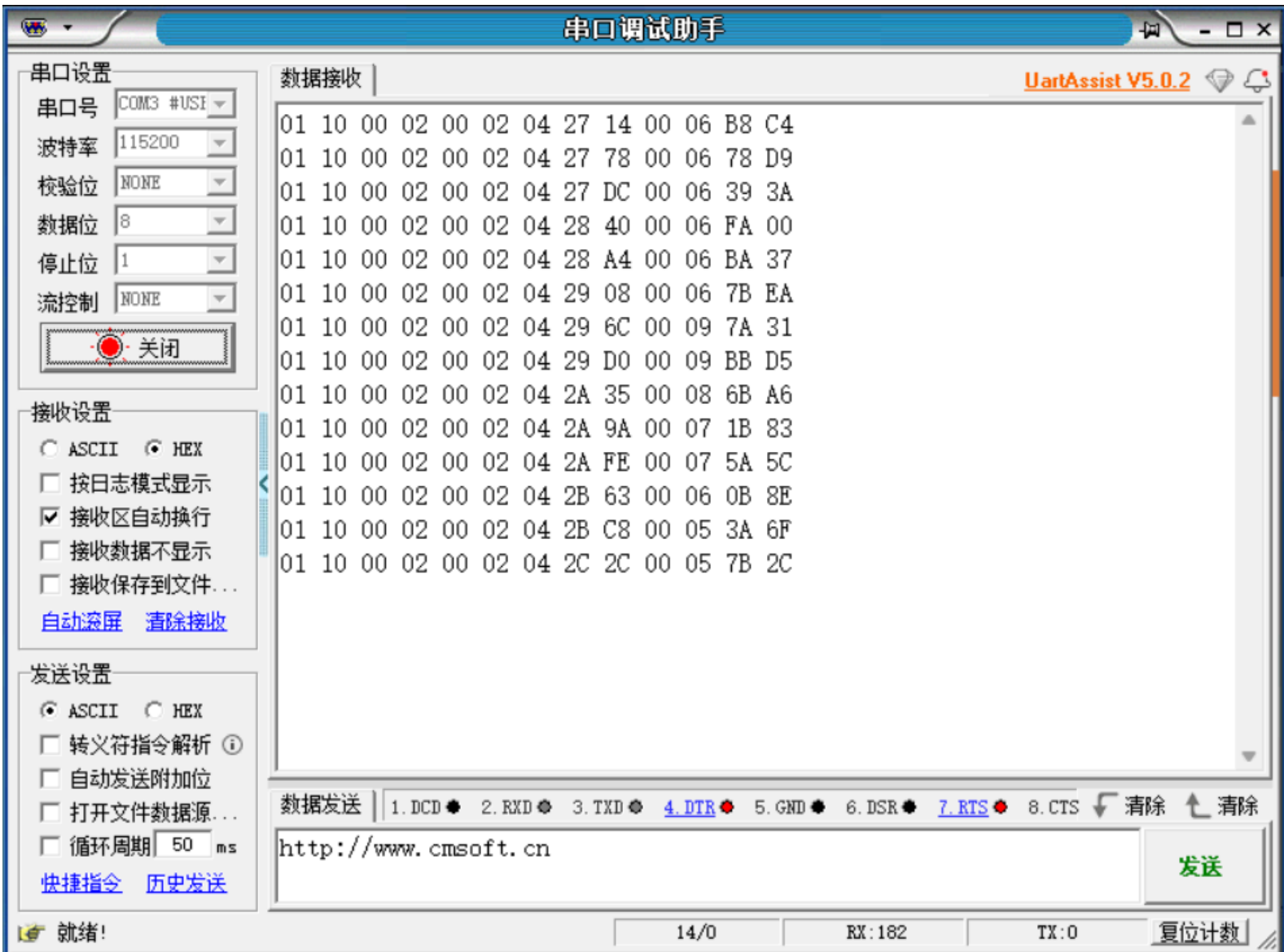
        error_code = embMasterReqwriteMultipleHoldingRegister(SLAVE_ADDR, /* salve address */
                                                                MB_SEND_REG_START, /* register start address */
                                                                MB_SEND_REG_NUM, /* register total number */
                                                                data, /* data to be written */
                                                                RT_WAITING_FOREVER); /* timeout */

        /* Record the number of errors */
        if (error_code != MB_MRE_NO_ERR)
        {
            error_count++;
            rt_kprintf("error_count=%d\n",error_count);
        }
        else
        {
            rt_kprintf("write slavle:%d ok\n",SLAVE_ADDR);
        }
    }
}
```

5. 运行结果

```
msh />mb_master_sample
msh />
```

打开串口调试助手，可以看到主机发送的写寄存器命令



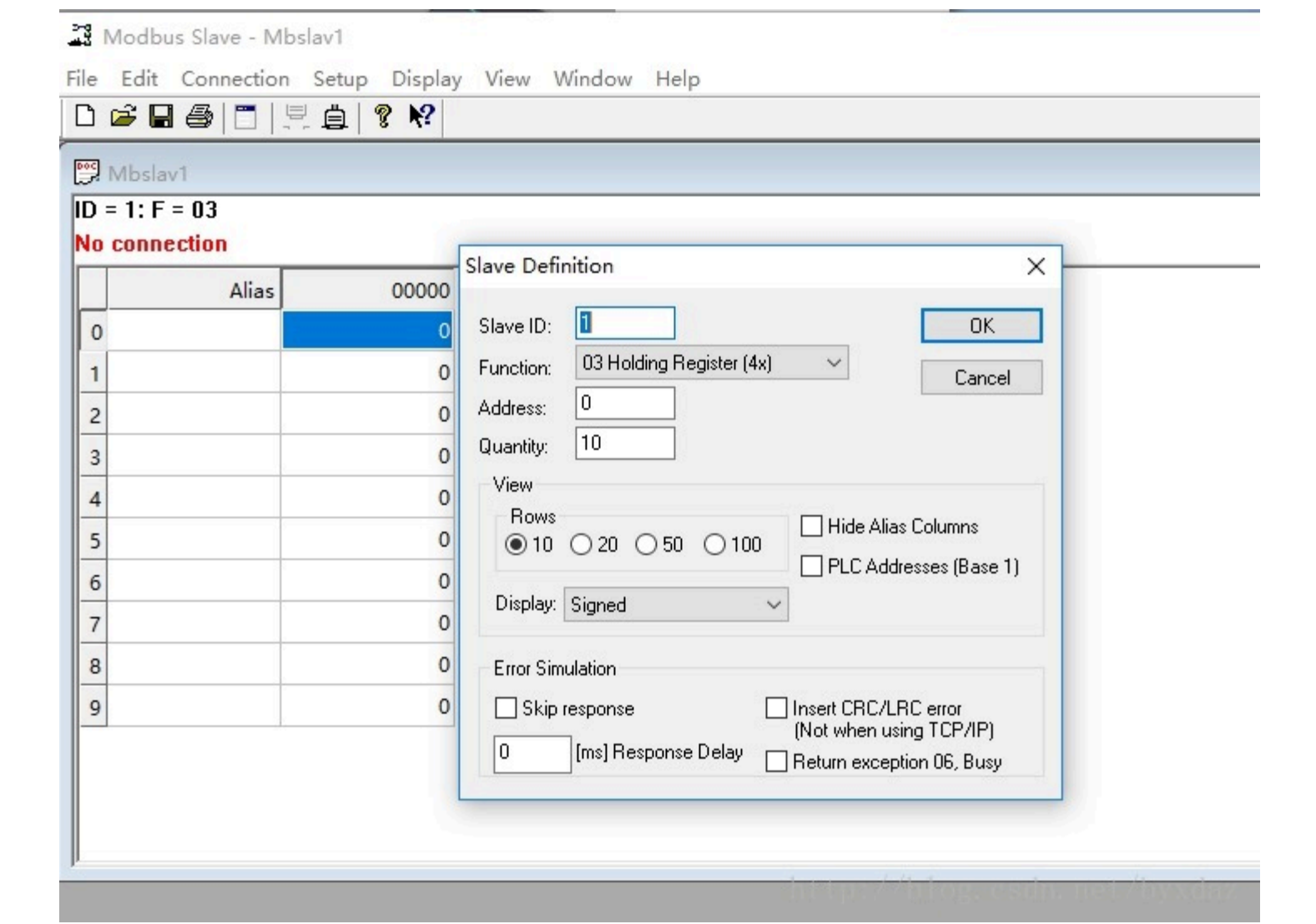
使用 modbus slave 从机测试。

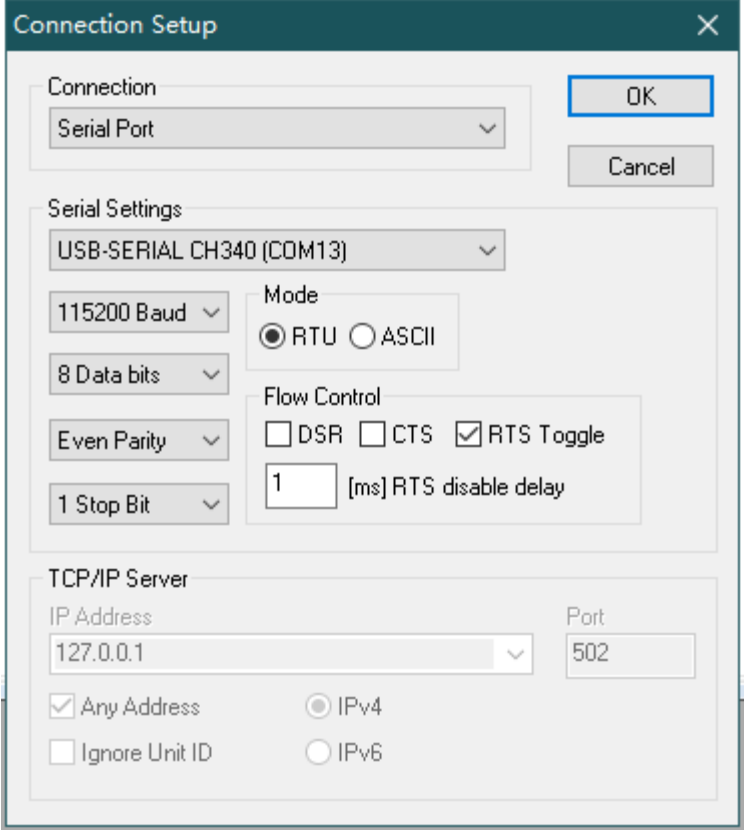
```
RT-Thread shell commands:
reboot          - Reboot System
mb_master_samlpe - run a modbus master sample
mb_slave_samlpe - run a modbus slave sample
version         - show RT-Thread version information
list_thread     - list thread
list_sem        - list semaphore in system
list_event      - list event in system
list_mutex      - list mutex in system
list_mailbox    - list mail box in system
list_msgqueue   - list message queue in system
list_mempool    - list memory pool in system
list_timer      - list timer in system
list_device     - list device in system
help            - RT-Thread shell help.
ps              - List threads in the system.
time           - Execute command with time.
free           - Show the memory usage in the system.
```

安装并打开 modbus slave 软件。

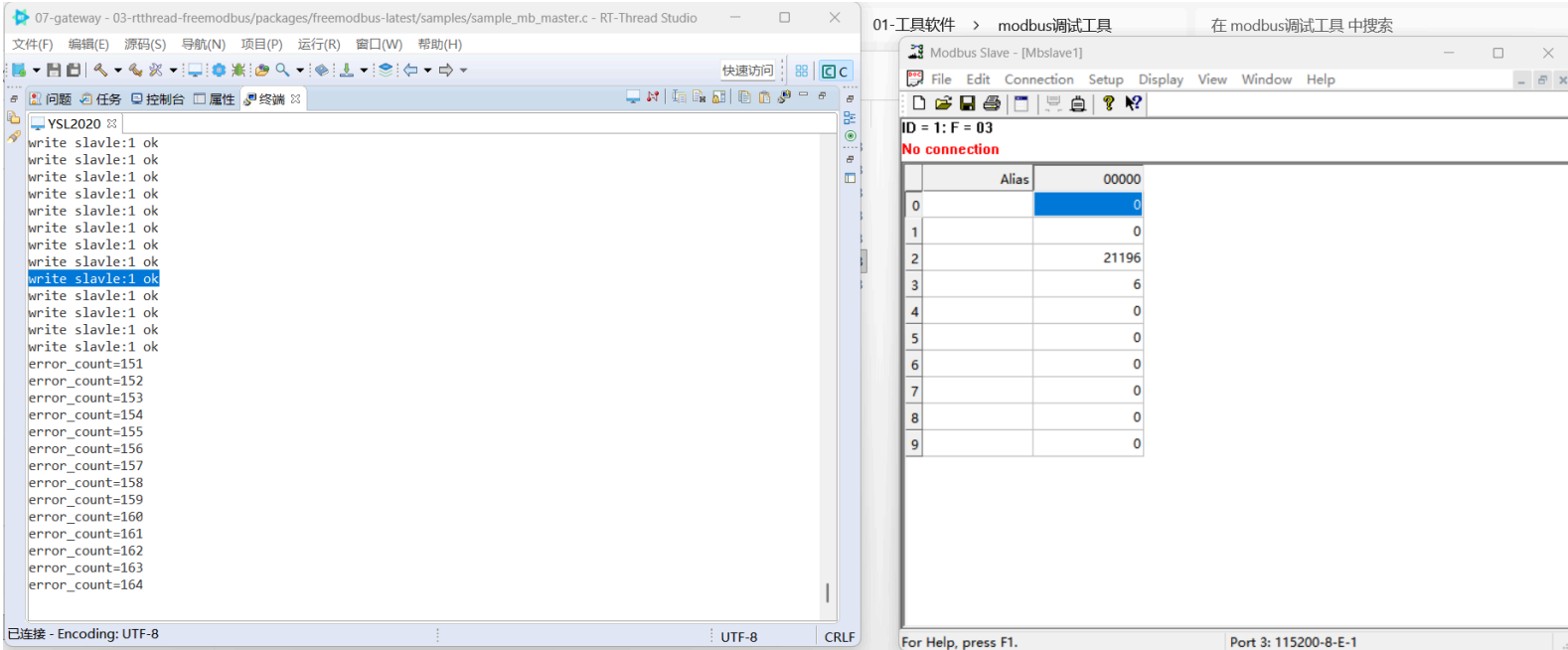
双击安装 STM32F407开发板V2\01-工具软件\modbus调试工具\MdbussSlaveSetup64Bit.exe 。

运行时需要输入 密钥的输入一下密钥， 密钥在同级目录内。





运行结果



1.4 练习04 Modbus + LVGL

1. 综合实战功能要求

- 通过485总线控制STM32F103V9的硬件。
- 支持通过485控制F103V9的LED1，LED2，BEEP，继电器。
- 支持通过485读取F103V9的温度，湿度。
- 支持通过485读取F103V9的电压，电流，功率。
- 使用LVGL控件显示出来。

2. Guider设计UI界面

- 设计界面



3. 程序设计

- 设置的手动切换状态

清除CHECKABLE标志，改为手动控制状态

```
/*
 * Copyright 2025 NXP
 * NXP Proprietary. This software is owned or controlled by NXP and may only be used strictly in
 * accordance with the applicable license terms. By expressly accepting such terms or by downloading,
 * installing,
 * activating and/or otherwise using the software, you are agreeing that you have read, and that you agree to
 * comply with and are bound by, such license terms. If you do not agree to be bound by the applicable
 * license
 * terms, then you may not retain, install, activate or otherwise use the software.
 */

#include "events_init.h"
#include <stdio.h>
#include "lvgl.h"

#include <rtthread.h>
#include <board.h>

#include "user_mb_app.h"

#define DBG_TAG "events_init"
#define DBG_LVL DBG_LOG
#include <rtdbg.h> // 头文件一定要在 #define DBG_LVL DBG_LOG 之后， 否则不输出

#if LV_USE_GUIIDER_SIMULATOR && LV_USE_FREEMASTER
#include "freemaster_client.h"
#endif

#define SLAVE_ADDR MB_SAMPLE_TEST_SLAVE_ADDR

extern rt_mailbox_t modbus_mb; // 消息通知， 消息邮箱
extern rt_ubase_t modbus_cmd; // 控制 modbus的命令
extern rt_ubase_t modbus_cmd_status; // 控制 modbus的命令
extern USHORT usmRegHo1dBuf[MB_MASTER_TOTAL_SLAVE_NUM][M_REG_HOLDING_NREGS];

static void screen_imgbtn_led1_event_handler(lv_event_t *e)
{
    lv_event_code_t code = lv_event_get_code(e);
    lv_obj_t *obj = lv_event_get_target(e);

    if (code == LV_EVENT_CLICKED)
    {
        // 获取当前状态（是否已经选中）
        bool current_state = lv_obj_has_state(obj, LV_STATE_CHECKED);
        // 期望的新状态是当前状态的取反
        bool new_state = !current_state;

        // 成功，则设置按钮为新状态
        if (new_state)
        {
            LOG_I("led1 on");
            if (rt_mb_send(modbus_mb, LED1ON) != RT_EOK)
            {
                LOG_E("modbus mailbox is full");
            }
        }
        else
        {
            LOG_I("led1 off");
            if (rt_mb_send(modbus_mb, LED1OFF) != RT_EOK)
            {
                LOG_E("modbus mailbox is full");
            }
        }
    }
}
```



```
static void update_device_cb(lv_timer_t * timer)
{
    static rt_uint8_t count1s = 0 ;
    char data[10] = {0} ;
    count1s ++ ;
    if(count1s >= 10)
    {
        count1s = 0 ;
        if (rt_mb_send(modbus_mb, SENSOR) != RT_EOK) // 发送通知
        {
            LOG_E("modbus mailbox is full");
        }
    }

    if(modbus_cmd_status == LED1ON)
    {
        lv_obj_add_state(guider_ui.screen_imgbtn_led1, LV_STATE_CHECKED);
    }
    else if(modbus_cmd_status == LED1OFF)
    {
        lv_obj_clear_state(guider_ui.screen_imgbtn_led1, LV_STATE_CHECKED);
    }
    else if(modbus_cmd_status == SENSOR)
    {
        rt_sprintf(data, "%d", usMRegHoldBuf[SLAVE_ADDR-1][1] );
        lv_label_set_text(guider_ui.screen_imgbtn_1_label, data);
    }
}

void events_init(lv_ui *ui)
{
    // 移除按钮的CHECKABLE标志, 改为手动控制状态
    lv_obj_clear_flag(ui->screen_imgbtn_led1, LV_OBJ_FLAG_CHECKABLE);

    // 添加事件监听
    lv_obj_add_event_cb(ui->screen_imgbtn_led1, screen_imgbtn_led1_event_handler, LV_EVENT_CLICKED, NULL);

    lv_timer_create(update_device_cb, 100, NULL);      // 中创建定时器

    // 选项1: 居中显示 (最常用)
    lv_obj_align(ui->screen_imgbtn_1_label, LV_ALIGN_BOTTOM_RIGHT, -12, -8);

}
```

移动 packages\freemodbus-latest\samples\sample_mb_master.c 到 applications 目录下

修改 sample_mb_master.c 为 mb_master.c 。

```
/*
 * Copyright (c) 2006-2022, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2019-06-21     flybreak     first version
 */

#include <rtthread.h>
#include <board.h>

#include "mb.h"
#include "mb_m.h"

#include "events_init.h"
#include <stdio.h>
#include "lvgl.h"

#define DBG_TAG "modbus"
// #define DBG_LVL DBG_LOG
#define DBG_LVL DBG_INFO
#include <rtdbg.h> // 头文件一定要在 #define DBG_LVL DBG_LOG 之后 , 否则不输出

#ifdef PKG_MODBUS_MASTER_SAMPLE
#define SLAVE_ADDR      MB_SAMPLE_TEST_SLAVE_ADDR
#define PORT_NUM        MB_MASTER_USING_PORT_NUM
#define PORT_BAUDRATE   MB_MASTER_USING_PORT_BAUDRATE
#else
#define SLAVE_ADDR      0x01
#define PORT_NUM        3
#define PORT_BAUDRATE   115200
#endif
#define PORT_PARITY      MB_PAR_NONE

#define MB_POLL_THREAD_PRIORITY  10
#define MB_SEND_THREAD_PRIORITY  RT_THREAD_PRIORITY_MAX - 1

#define MB_SEND_REG_START  0
#define MB_SEND_REG_NUM    5

#define MB_POLL_CYCLE_MS   50

__IO rt_mailbox_t modbus_mb ;           // 消息通知, 消息邮箱 ,
rt_ubase_t modbus_cmd;                 // 控制 modbus的命令
__IO rt_ubase_t modbus_cmd_status;     // 控制 modbus的命令
__IO rt_uint8_t modbus_sensor_flag = 0; // 读传感器标志位
__IO USHORT modbus_send_data[10] = {0};

static void send_thread_entry(void *parameter)
{
    eMBMasterReqErrCode error_code = MB_MRE_NO_ERR;

    while (1)
    {
        if( rt_mb_rcv(modbus_mb, &modbus_cmd, RT_WAITING_FOREVER) == RT_EOK)
        {
            LOG_D("modbus_cmd=%d", modbus_cmd) ;
            if(modbus_cmd == LED1ON) // LED1 ON
            {
                modbus_send_data[0] = modbus_send_data[0] | ( 0x1<< 0 ) ;
                error_code = eMBMasterReqWriteHoldingRegister(SLAVE_ADDR,
                                                                MB_SEND_REG_START, modbus_send_data[0], 1000);

                if (error_code == MB_MRE_NO_ERR)
                {
                    modbus_cmd_status = LED1ON ;
                }
                else
                {
                    LOG_I("modbus slave %d not found", SLAVE_ADDR) ;
                }
            }
            else if(modbus_cmd == LED1OFF) // LED1 OFF
            {
                modbus_send_data[0] = modbus_send_data[0] & ( ~ ( 0x1<< 0 ) );
                error_code = eMBMasterReqWriteHoldingRegister(SLAVE_ADDR,
                                                                MB_SEND_REG_START, modbus_send_data[0], 1000);

                if (error_code == MB_MRE_NO_ERR)
                {
                    modbus_cmd_status = LED1OFF ;
                }
                else
                {
                    LOG_I("modbus slave %d not found", SLAVE_ADDR) ;
                }
            }
            else if(modbus_cmd == SENSOR) // 读传感器
            {
                error_code = eMBMasterReqReadHoldingRegister(SLAVE_ADDR,
                                                                MB_SEND_REG_START+1, MB_SEND_REG_NUM, 1000);

                if (error_code == MB_MRE_NO_ERR)
                {
                    modbus_cmd_status = SENSOR ;
                }
                else
                {
                    LOG_I("modbus slave %d not found", SLAVE_ADDR) ;
                }
            }
        }
    }
}
```

```
    }
}

}

static void mb_master_poll(void *parameter)
{
    eMBMasterInit(MB_RTU, PORT_NUM, PORT_BAUDRATE, PORT_PARITY);
    eMBMasterEnable();

    // 动态创建邮箱
    modbus_mb = rt_mb_create("modbus", 8, RT_IPC_FLAG_FIFO);
    if(modbus_mb == RT_NULL)
    {
        LOG_E("rt_mb_create failed! ...") ;
    }

    while (1)
    {
        eMBMasterPoll();
        rt_thread_mdelay(MB_POLL_CYCLE_MS);
    }
}

static int mb_master(void)
{
    static rt_uint8_t is_init = 0;
    rt_thread_t tid1 = RT_NULL, tid2 = RT_NULL;

    if (is_init > 0)
    {
        rt_kprintf("sample is running\n");
        return -RT_ERROR;
    }
    tid1 = rt_thread_create("md_m_poll", mb_master_poll, RT_NULL, 512, MB_POLL_THREAD_PRIORITY, 10);
    if (tid1 != RT_NULL)
    {
        rt_thread_startup(tid1);
    }
    else
    {
        goto __exit;
    }
    tid2 = rt_thread_create("md_m_send", send_thread_entry, RT_NULL, 512, MB_SEND_THREAD_PRIORITY - 2, 10);
    if (tid2 != RT_NULL)
    {
        rt_thread_startup(tid2);
    }
    else
    {
        goto __exit;
    }

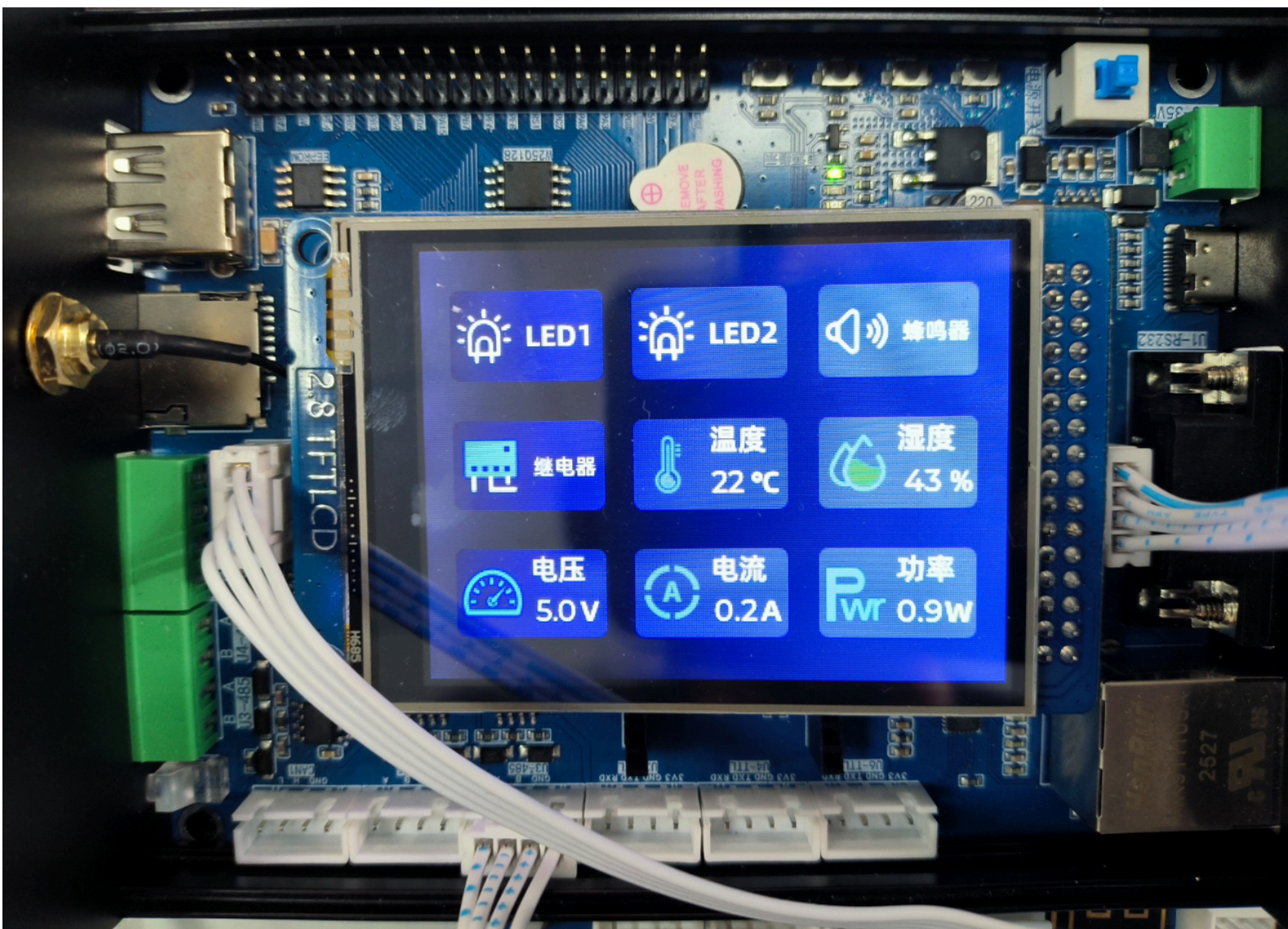
    is_init = 1;
    return RT_EOK;

__exit:
    if (tid1)
        rt_thread_delete(tid1);
    if (tid2)
        rt_thread_delete(tid2);

    return -RT_ERROR;
}

INIT_ENV_EXPORT(mb_master);
```

4. 运行结果



第02章 网关控制器OTA在线升级

2.1 练习 05 RT-Thread bootloader

1. OTA原理

OTA（空中下载技术）是通过移动通信的空中接口实现对移动终端设备进行远程管理和升级的技术。我们调试硬件设备的时候通常是通过数据线进行固件的烧录，如果是在办公室的环境下这种方式是很方便的，但是如果设备距离较远，有线的方式就有很大的局限性。因此，远程升级技术就显得很重要了。

RT-thread有一套OTA的方案可以参考，今天主要就介绍这套方案的使用以及优缺点。

2. RT-thread Bootloader

我这边主要介绍的是RT-Thread开发团队提供的通用Bootloader。

Bootloader的功能前面也有介绍了，它作为一个引导程序，更新app分区中的固件或者跳转到app应用程序。

Bootloader可以通过网页端在线生成的方式来获取。开发者根据自己使用的芯片，填写相关参数，然后点击生成按钮，即可在线生成Bootloader。

RT-thread官方 Bootloader 步骤：[STM32通用Bootloader](#) 。

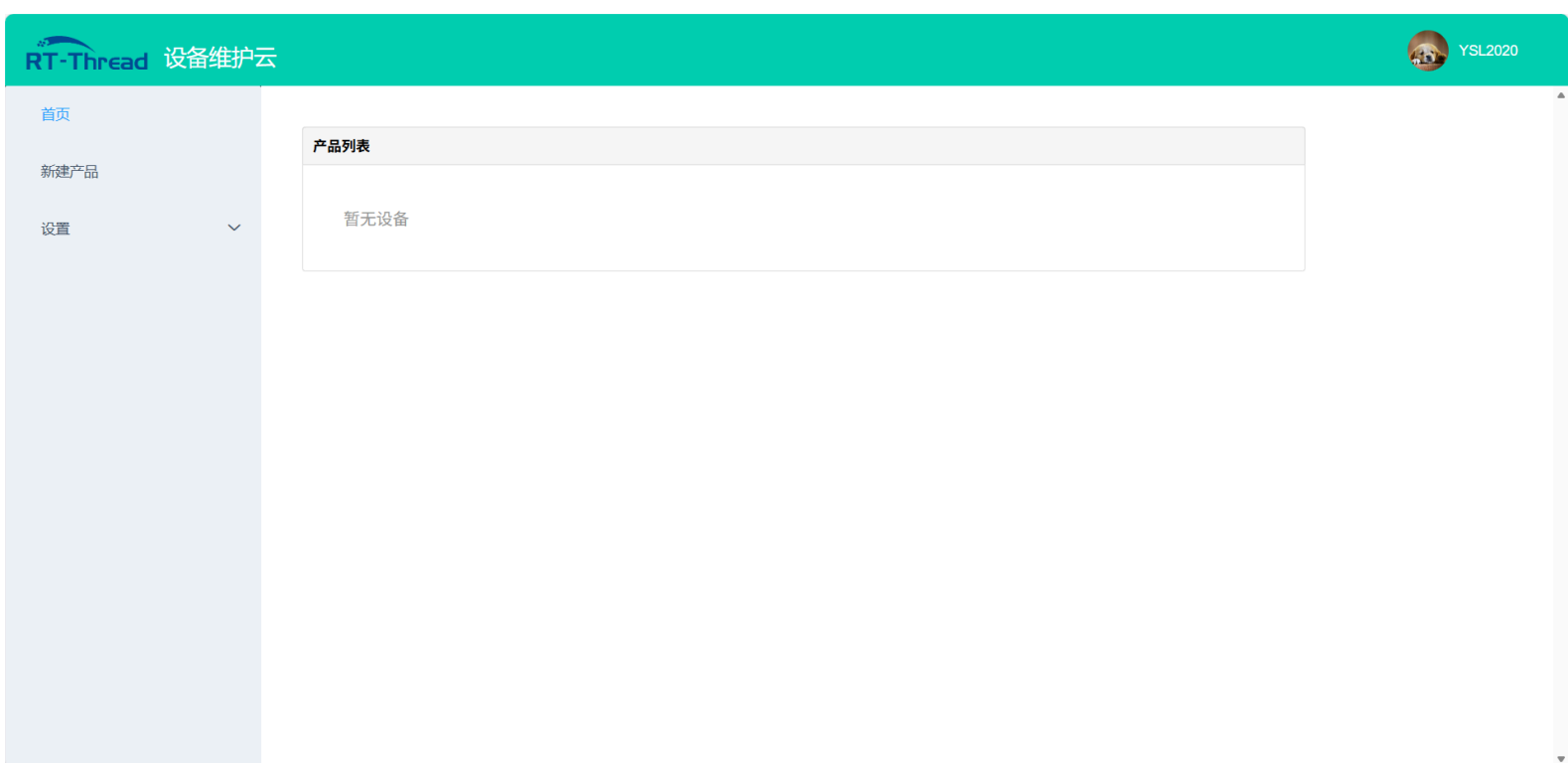
Bootloader 在线获取地址：<http://iot.rt-thread.com>

3. 生成Bootloader

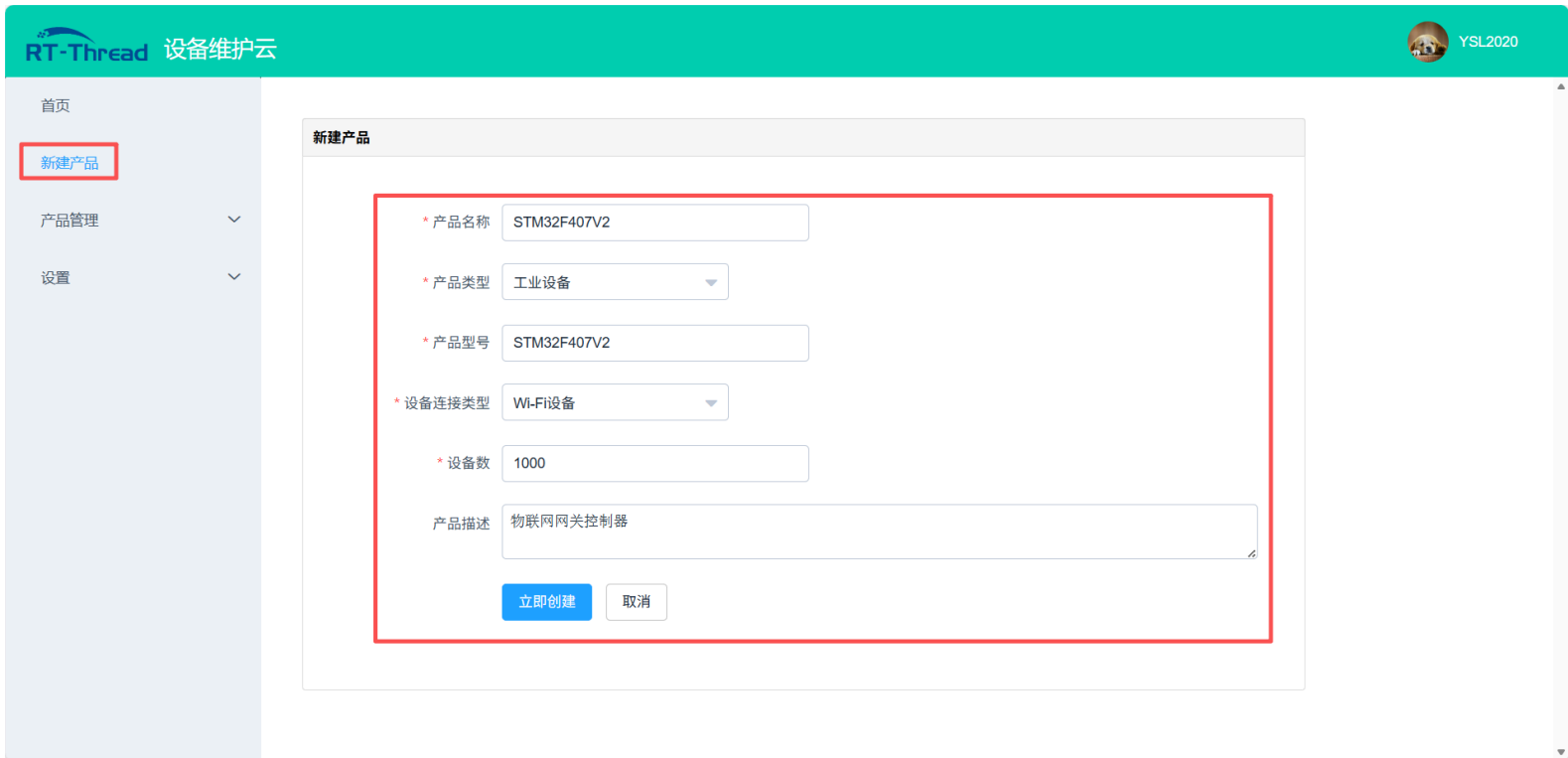
- 注册并登录账号



• 新建 Bootloader

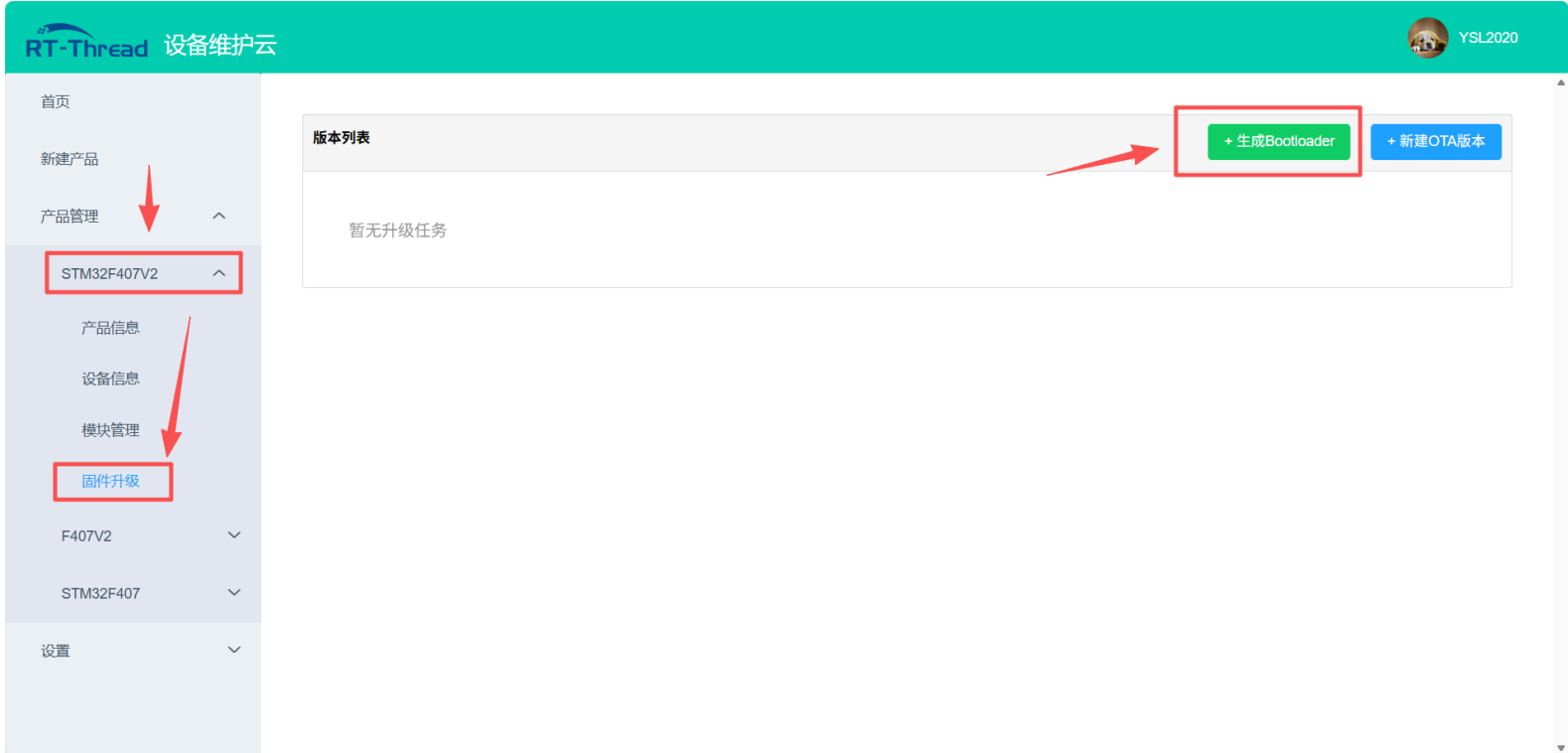


• 新建产品



立即创建，即可创建设备。

• 生成bootloader



• 配置硬件信息



• 分区表配置



• 配置邮箱



- 保存配置并生成bootloader

加密压缩

☐ 固件使用AES256加密

☐ 固件压缩

logo和邮箱配置

☐ 更换logo

* 请输入邮箱:

请输入正确的邮箱

保存当前页面 Bootloader 配置

导入 Bootloader 配置

导出 Bootloader 配置

点击生成Bootloader

- 等待生成完成

logo和邮箱配置

☐ 更换logo

正在生成Bootloader,请等待.....

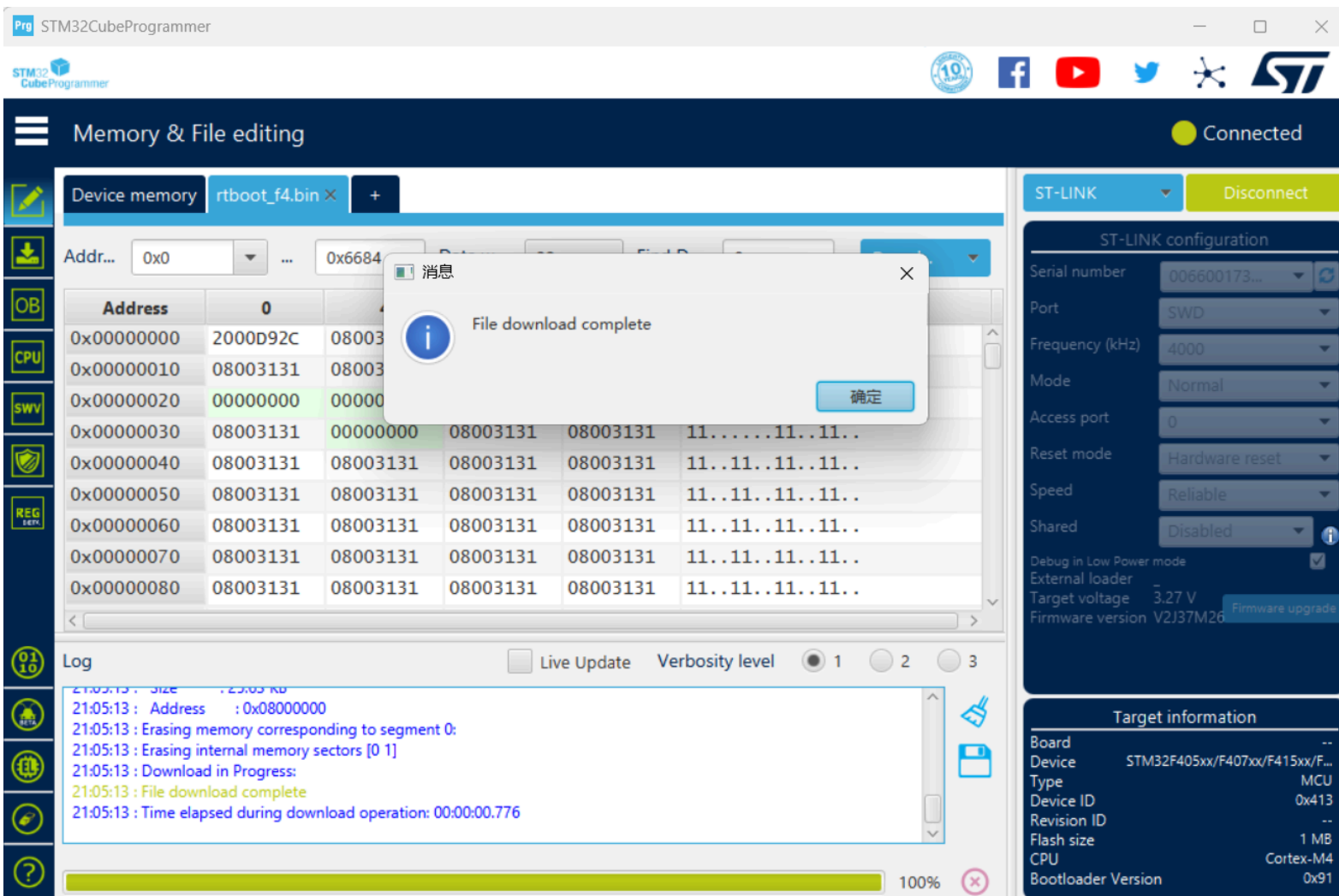
shengli001842@qq.com

* 请输入邮箱:

正在生成Bootloader, 请等待

- 下载程序

下载 `rtboot_f4.bin` 文件到芯片内。



4. 运行结果

bootloader启动程序，因为没有应用程序，所有会有错误，这个是正常的。

```
[SFUD]Find a winbond flash chip. Size is 16777216 bytes.
[SFUD]spi flash device is initialize success.

_ _ _ _ _
|_) | _ |_) / \ / \ |
| \ | |_) \ / \ / |

2006 - 2019 Copyright by rt-thread team 0.9.2 build Nov 9 2025

[D/FAL] (fal_flash_init:63) Flash device | onchip_flash_16k_part | addr: 0x08
[D/FAL] (fal_flash_init:63) Flash device | onchip_flash | addr: 0x08
[D/FAL] (fal_flash_init:63) Flash device | nor_flash | addr: 0x00

[I/FAL] | name | flash_dev | offset | length |
[I/FAL] | app | onchip_flash | 0x00000000 | 0x000e0000 |
[I/FAL] | download | nor_flash | 0x00000000 | 0x00100000 |
[I/FAL] | factory | nor_flash | 0x00100000 | 0x00100000 |

[I/FAL] System initialization successful.

[I]RT-Thread OTA package(V0.2.1) initialize success.
[E]Get firmware header occur CRC32(cal.c.crc: 7b93c5c8 != hdr.info_crc32: ffffffff
[E]Get OTA download partition firmware header failed!
[E]Get firmware header occur CRC32(cal.c.crc: 7b93c5c8 != hdr.info_crc32: ffffffff
[I]Begin to execute the program on app partition.
[E/FAL] Can't find user firmware on app partition. (Address: 0x08020000, Data: 0
[E/FAL] Bootloader stop.
```

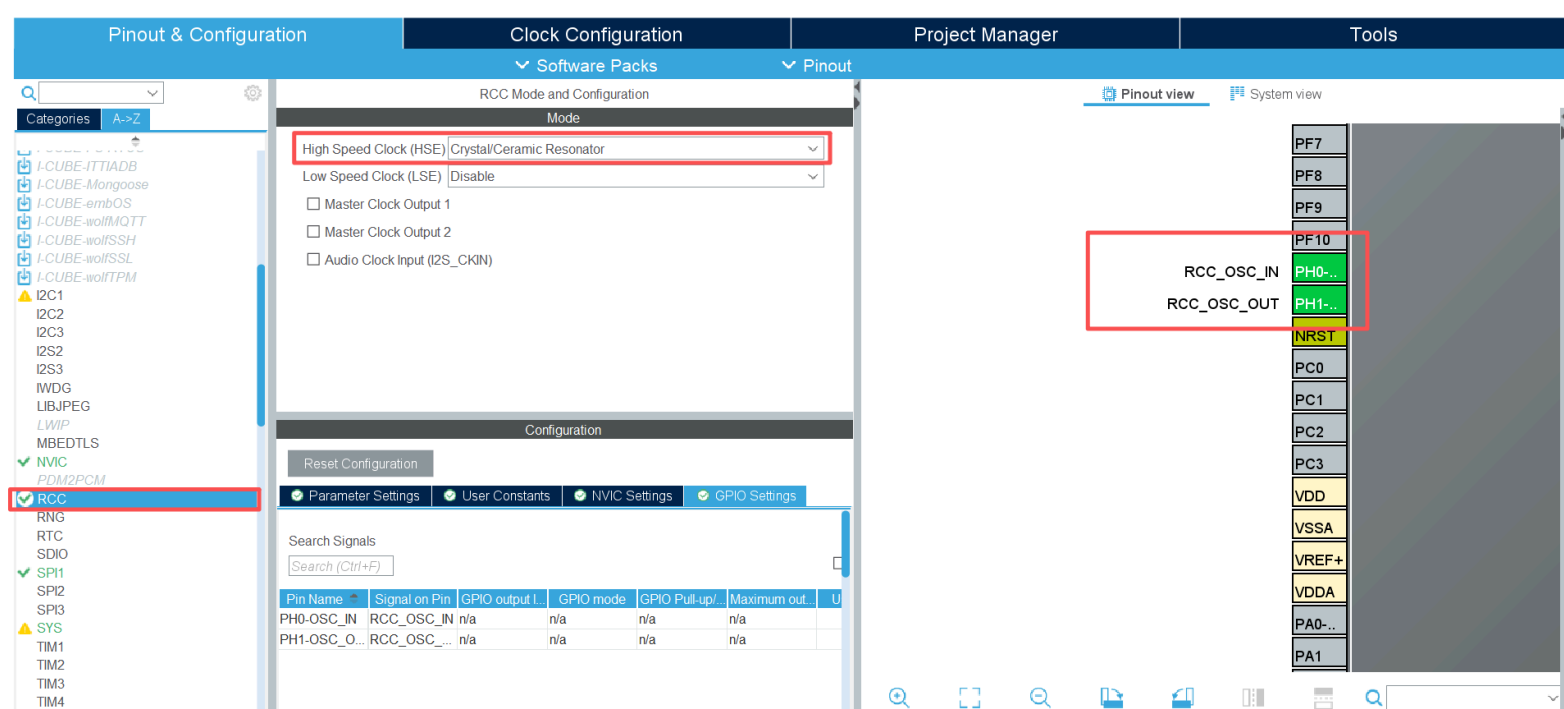
2.2 练习 06 RT-Thread OTA

1. 题目要求

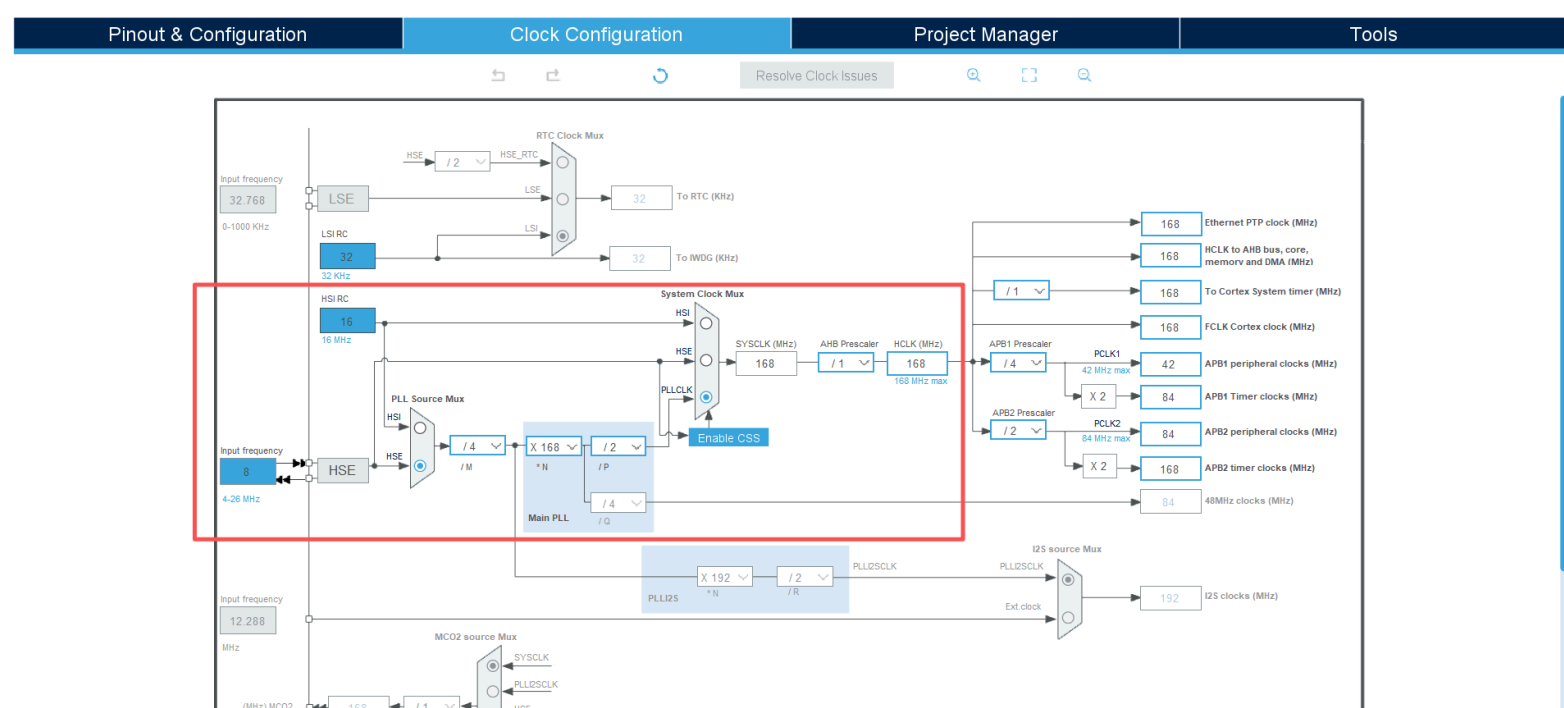
- 新建工程，从头开始配置工程。
- 支持Ymodem协议下载固件并升级系统。

2. 配置 CubeMX

- 配置时钟



- 配置时钟树



● 配置SWD接口



3. 配置 RT-Thread Settings

- 启用一下配置



复制rt-thread\components\fa\samples\porting\fa_cfg.h到rt-thread\components\fa\inc 目录下

- 编译后报错

```
Linking...
./rt-thread/components/fal/src/fal_flash.o(.rodata.device_table+0x0): undefined reference to
`stm32f2_onchip_flash'
collect2.exe: error: ld returned 1 exit status
make: *** [makefile:69: rtthread.elf] Error 1
"make -j32 all" terminated with exit code 2. Build might be incomplete.
```

修改fal分区，保持和bootloader分区保持一致

rt-thread\components\fal\inc\fal_cfg.h

```
/*
 * Copyright (c) 2006-2018, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date       Author       Notes
 * 2018-05-17   armink       the first version
 */

#ifndef _FAL_CFG_H_
#define _FAL_CFG_H_

#include <rtconfig.h>
#include <board.h>

#define NOR_FLASH_DEV_NAME            "norflash0"

/* ===== Flash device Configuration ===== */
extern const struct fal_flash_dev stm32_onchip_flash_128k;
extern struct fal_flash_dev nor_flash0;

/* flash device table */
#define FAL_FLASH_DEV_TABLE
{
    &stm32_onchip_flash_128k,
    &nor_flash0,
}
/* ===== Partition Configuration ===== */
#ifdef FAL_PART_HAS_TABLE_CFG
/* partition table */
#define FAL_PART_TABLE
{
    {FAL_PART_MAGIC_WORD,    "app",    "onchip_flash",    0,    896*1024, 0}, \
    {FAL_PART_MAGIC_WORD,    "download", NOR_FLASH_DEV_NAME,    0,    1024*1024, 0}, \
    {FAL_PART_MAGIC_WORD,    "factory", NOR_FLASH_DEV_NAME,    1*1024*1024,    1024*1024, 0}, \
    {FAL_PART_MAGIC_WORD,    "env",    NOR_FLASH_DEV_NAME,    2*1024*1024,    1024*1024, 0}, \
    {FAL_PART_MAGIC_WORD,    "filesys", NOR_FLASH_DEV_NAME,    3*1024*1024, 4*1024*1024, 0}, \
    {FAL_PART_MAGIC_WORD,    "eflash", NOR_FLASH_DEV_NAME,    7*1024*1024, 9*1024*1024, 0}, \
}
#endif /* FAL_PART_HAS_TABLE_CFG */

#endif /* _FAL_CFG_H_ */
```

启用 board.h 中的宏值

```
/** if you want to use spi bus you can use the following instructions.
 *
 * STEP 1, open spi driver framework support in the RT-Thread Settings file
 *
 * STEP 2, define macro related to the spi bus
 *          such as      #define BSP_USING_SPI1
 *
 * STEP 3, copy your spi init function from stm32xxxx_hal_msp.c generated by stm32cubemx to the end of
board.c file
 *          such as      void HAL_SPI_MspInit(SPI_HandleTypeDef* hspi)
 *
 * STEP 4, modify your stm32xxxx_hal_config.h file to support spi peripherals. define macro related to the
peripherals
 *          such as      #define HAL_SPI_MODULE_ENABLED
 */

#define BSP_USING_SPI1
/*#define BSP_USING_SPI2*/
/*#define BSP_USING_SPI3*/

/*----- SPI CONFIG END -----*/

/*----- ON_CHIP_FLASH CONFIG BEGIN -----*/

/** if you want to use on chip flash you can use the following instructions.
 *
 * STEP 1 define macro related to the on chip flash
 *          such as      BSP_USING_ON_CHIP_FLASH
 *
 * STEP 2, modify your stm32xxxx_hal_config.h file to support on chip flash peripherals. define macro
related to the peripherals
 *          such as      #define HAL_FLASH_MODULE_ENABLED
 *
 */

#define BSP_USING_ON_CHIP_FLASH

/*----- ON_CHIP_FLASH CONFIG END -----*/
```

- 编译错误

```
arm-none-eabi-gcc "../cubemx/Src/stm32f4xx_hal_msp.c"
arm-none-eabi-gcc "../applications/main.c"
../drivers/drv_flash_f4.c:376:75: error: 'STM32_FLASH_START_ADRESS_16K' undeclared here (not in a function)
const struct fal_flash_dev stm32_onchip_flash_16k = { "onchip_flash_16k", STM32_FLASH_START_ADRESS_16K,
FLASH_SIZE_GRANULARITY_16K, (16 * 1024), {NULL, fal_flash_read_16k, fal_flash_write_16k,
fal_flash_erase_16k} };
^
../drivers/drv_flash_f4.c:376:105: error: 'FLASH_SIZE_GRANULARITY_16K' undeclared here (not in a function)
const struct fal_flash_dev stm32_onchip_flash_16k = { "onchip_flash_16k", STM32_FLASH_START_ADRESS_16K,
FLASH_SIZE_GRANULARITY_16K, (16 * 1024), {NULL, fal_flash_read_16k, fal_flash_write_16k,
fal_flash_erase_16k} };
^
../drivers/drv_flash_f4.c:377:75: error: 'STM32_FLASH_START_ADRESS_64K' undeclared here (not in a function)
const struct fal_flash_dev stm32_onchip_flash_64k = { "onchip_flash_64k", STM32_FLASH_START_ADRESS_64K,
FLASH_SIZE_GRANULARITY_64K, (64 * 1024), {NULL, fal_flash_read_64k, fal_flash_write_64k,
fal_flash_erase_64k} };
^
../drivers/drv_flash_f4.c:377:105: error: 'FLASH_SIZE_GRANULARITY_64K' undeclared here (not in a function)
const struct fal_flash_dev stm32_onchip_flash_64k = { "onchip_flash_64k", STM32_FLASH_START_ADRESS_64K,
FLASH_SIZE_GRANULARITY_64K, (64 * 1024), {NULL, fal_flash_read_64k, fal_flash_write_64k,
fal_flash_erase_64k} };
^
../drivers/drv_flash_f4.c:378:77: error: 'STM32_FLASH_START_ADRESS_128K' undeclared here (not in a function)
const struct fal_flash_dev stm32_onchip_flash_128k = { "onchip_flash_128k", STM32_FLASH_START_ADRESS_128K,
FLASH_SIZE_GRANULARITY_128K, (128 * 1024), {NULL, fal_flash_read_128k, fal_flash_write_128k,
fal_flash_erase_128k} };
^
../drivers/drv_flash_f4.c:378:108: error: 'FLASH_SIZE_GRANULARITY_128K' undeclared here (not in a function)
const struct fal_flash_dev stm32_onchip_flash_128k = { "onchip_flash_128k", STM32_FLASH_START_ADRESS_128K,
FLASH_SIZE_GRANULARITY_128K, (128 * 1024), {NULL, fal_flash_read_128k, fal_flash_write_128k,
fal_flash_erase_128k} };
^
make: *** [drivers/subdir.mk:99: drivers/drv_flash_f4.o] Error 1
make: *** waiting for unfinished jobs....
"make -j32 all" terminated with exit code 2. Build might be incomplete.

22:04:31 Build Failed. 8 errors, 0 warnings. (took 3s.446ms)
```

这个需要根据407ZG芯片手册去调整。

修改文件 drivers\drv_flash_f4.c

```
/* Base address of the Flash sectors Bank 1 */
#define ADDR_FLASH_SECTOR_0 ((uint32_t)0x08000000) /* Base @ of Sector 0, 16 kbytes */
#define ADDR_FLASH_SECTOR_1 ((uint32_t)0x08004000) /* Base @ of Sector 1, 16 kbytes */
#define ADDR_FLASH_SECTOR_2 ((uint32_t)0x08008000) /* Base @ of Sector 2, 16 kbytes */
#define ADDR_FLASH_SECTOR_3 ((uint32_t)0x0800C000) /* Base @ of Sector 3, 16 kbytes */
#define ADDR_FLASH_SECTOR_4 ((uint32_t)0x08010000) /* Base @ of Sector 4, 64 kbytes */
#define ADDR_FLASH_SECTOR_5 ((uint32_t)0x08020000) /* Base @ of Sector 5, 128 kbytes */
#define ADDR_FLASH_SECTOR_6 ((uint32_t)0x08040000) /* Base @ of Sector 6, 128 kbytes */
#define ADDR_FLASH_SECTOR_7 ((uint32_t)0x08060000) /* Base @ of Sector 7, 128 kbytes */
#define ADDR_FLASH_SECTOR_8 ((uint32_t)0x08080000) /* Base @ of Sector 8, 128 kbytes */
#define ADDR_FLASH_SECTOR_9 ((uint32_t)0x080A0000) /* Base @ of Sector 9, 128 kbytes */
#define ADDR_FLASH_SECTOR_10 ((uint32_t)0x080C0000) /* Base @ of Sector 10, 128 kbytes */
#define ADDR_FLASH_SECTOR_11 ((uint32_t)0x080E0000) /* Base @ of Sector 11, 128 kbytes */
```



```
/* Base address of the Flash sectors Bank 2 */
#define ADDR_FLASH_SECTOR_12 ((uint32_t)0x08100000) /* Base @ of Sector 0, 16 Kbytes */
#define ADDR_FLASH_SECTOR_13 ((uint32_t)0x08104000) /* Base @ of Sector 1, 16 Kbytes */
#define ADDR_FLASH_SECTOR_14 ((uint32_t)0x08108000) /* Base @ of Sector 2, 16 Kbytes */
#define ADDR_FLASH_SECTOR_15 ((uint32_t)0x0810C000) /* Base @ of Sector 3, 16 Kbytes */
#define ADDR_FLASH_SECTOR_16 ((uint32_t)0x08110000) /* Base @ of Sector 4, 64 Kbytes */
#define ADDR_FLASH_SECTOR_17 ((uint32_t)0x08120000) /* Base @ of Sector 5, 128 Kbytes */
#define ADDR_FLASH_SECTOR_18 ((uint32_t)0x08140000) /* Base @ of Sector 6, 128 Kbytes */
#define ADDR_FLASH_SECTOR_19 ((uint32_t)0x08160000) /* Base @ of Sector 7, 128 Kbytes */
#define ADDR_FLASH_SECTOR_20 ((uint32_t)0x08180000) /* Base @ of Sector 8, 128 Kbytes */
#define ADDR_FLASH_SECTOR_21 ((uint32_t)0x081A0000) /* Base @ of Sector 9, 128 Kbytes */
#define ADDR_FLASH_SECTOR_22 ((uint32_t)0x081c0000) /* Base @ of Sector 10, 128 Kbytes */
#define ADDR_FLASH_SECTOR_23 ((uint32_t)0x081E0000) /* Base @ of Sector 11, 128 Kbytes */
```

//起始地址 增加的内容

```
#define STM32_FLASH_START_ADDRESS_16K ADDR_FLASH_SECTOR_0
#define STM32_FLASH_START_ADDRESS_64K ADDR_FLASH_SECTOR_4
#define STM32_FLASH_START_ADDRESS_128K ADDR_FLASH_SECTOR_5
```

//大小

```
#define FLASH_SIZE_GRANULARITY_16K (4*16*1024) //4个16K
#define FLASH_SIZE_GRANULARITY_64K ( 64*1024) //1个64K
#define FLASH_SIZE_GRANULARITY_128K (7*128*1024) //7个128K
```

```
const struct fal_flash_dev stm32_onchip_flash_16k = { "onchip_flash_16k", STM32_FLASH_START_ADDRESS_16K,
FLASH_SIZE_GRANULARITY_16K, (16 * 1024), {NULL, fal_flash_read_16k, fal_flash_write_16k,
fal_flash_erase_16k} };
const struct fal_flash_dev stm32_onchip_flash_64k = { "onchip_flash_64k", STM32_FLASH_START_ADDRESS_64K,
FLASH_SIZE_GRANULARITY_64K, (64 * 1024), {NULL, fal_flash_read_64k, fal_flash_write_64k,
fal_flash_erase_64k} };
const struct fal_flash_dev stm32_onchip_flash_128k = { "onchip_flash", STM32_FLASH_START_ADDRESS_128K,
FLASH_SIZE_GRANULARITY_128K, (128 * 1024), {NULL, fal_flash_read_128k, fal_flash_write_128k,
fal_flash_erase_128k} };
```

- 原理如下：

主存储器一共12个扇区，4个16K、1个64K、7个128K;

表 5. Flash 模块构成（STM32F40x 和 STM32F41x）

块	名称	块基址	大小
主存储器	扇区 0	0x0800 0000 - 0x0800 3FFF	16 KB
	扇区 1	0x0800 4000 - 0x0800 7FFF	16 KB
	扇区 2	0x0800 8000 - 0x0800 BFFF	16 KB
	扇区 3	0x0800 C000 - 0x0800 FFFF	16 KB
	扇区 4	0x0801 0000 - 0x0801 FFFF	64 KB
	扇区 5	0x0802 0000 - 0x0803 FFFF	128 KB
	扇区 6	0x0804 0000 - 0x0805 FFFF	128 KB
	· · ·	· · ·	· · ·
	扇区 11	0x080E 0000 - 0x080F FFFF	128 KB
系统存储器		0x1FFF 0000 - 0x1FFF 77FF	30 KB
OTP 区域		0x1FFF 7800 - 0x1FFF 7A0F	528 字节
选项字节		0x1FFF C000 - 0x1FFF C00F	16 字节

编译程序， 可以成功编译

```
arm-none-eabi-gcc "../drivers/drv_usbd.c"
arm-none-eabi-gcc "../drivers/drv_usbh.c"
arm-none-eabi-gcc "../drivers/drv_wdt.c"
arm-none-eabi-gcc "../cubemx/src/main.c"
arm-none-eabi-gcc "../cubemx/src/stm32f4xx_hal_msp.c"
arm-none-eabi-gcc "../applications/main.c"
linking...
arm-none-eabi-objcopy -O binary "rtthread.elf" "rtthread.bin"
arm-none-eabi-size --format=berkeley "rtthread.elf"
text    data    bss     dec      hex filename
135800   2144    4376   142320   22bf0 rtthread.elf

               Used Size(B)              Used Size(KB)
Flash:         137944 B                    134.71 KB
RAM:           6520 B                      6.37 KB

22:11:18 Build Finished. 0 errors, 0 warnings. (took 3s.928ms)
```

- 修改 link.lids

linkscripts\STM32F407ZGlink.lids

可用存储区域

名称	起始地址	大小 (kB)	
ROM (rx)	0x08020000	896	
RAM (rw)	0x20000000	128	

添加存储区域

栈大小

0x400|

段到存储区域的映射关系

段名	存储区域	
.bss	RAM	
.ARM.exidx	ROM	
.stack	RAM	
.text	ROM	
.data	RAM	

- 修改向量表偏移

libraries\CMSIS\Device\ST\STM32F4xx\Source\Templates\system_stm32f4xx.c

```
/*!< Uncomment the following line if you need to relocate your vector Table in
Internal SRAM. */
/* #define VECT_TAB_SRAM */
#define VECT_TAB_OFFSET 0x20000 /*!< Vector Table base offset field.
This value must be a multiple of 0x200. */
/*****
```

编译工程。

5. 源代码

在 applications 目录内， 新建 fal_app.c 文件

applications\fal_app.c

```
/*
 * Copyright (c) 2006-2021, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2025-11-09     shengli      the first version
 */

#include <rtthread.h>
#include <board.h>
#include <drv_spi.h>
#include <rtddef.h>
#include <fal.h> // 必须包含FAL头文件
#include <spi_flash.h>
#include "spi_flash_sfud.h"

rt_spi_flash_device_t w25q128; // 定义 w25q128 结构体

int spiflash_device_init(void)
{
    //注册SPI设备
    rt_hw_spi_device_attach("spi1", "SPI1", GPIOG, GPIO_PIN_15);
    //挂载flash
    if((w25q128 = rt_sfud_flash_probe(FAL_USING_NOR_FLASH_DEV_NAME, "SPI1"))== NULL)
    {
        rt_kprintf("Error! W25Q128FV NOT FOUND!");
    }
    return RT_EOK;
}

static int fal_app(void)
{
    spiflash_device_init();
```

```
fal_init();
    return RT_EOK ;
}

/* 开机自动启动 */
INIT_DEVICE_EXPORT(fal_app);
```

applications\main.c

```
/*
 * Copyright (c) 2006-2025, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2025-11-09      RT-Thread    first version
 */

#include <rtthread.h>

#define DBG_TAG "main"
#define DBG_LVL DBG_LOG
#include <rtdbg.h>

#define APP_VERSION "1.1.0"

int main(void)
{
    int count = 1;
    LOG_D("APP version: %s build %s\r\n", APP_VERSION, __DATE__);
    while (count++)
    {
        rt_thread_mdelay(1000);
    }

    return RT_EOK;
}
```

6. 运行APP

下载程序，运行结果如下：

```
[SFUD]Find a winbond flash chip. Size is 16777216 bytes.
[SFUD]spi flash device is initialize success.

  _ _ _ _ _
|_) | _ |_) / \ / \ |
| \ | _ |_) \_/ \_/ |
2006 - 2019 Copyright by rt-thread team
          0.9.2 build Nov  9 2025
[D/FAL] (fal_flash_init:63) Flash device | onchip_flash_16k_part | addr: 0x08000000 | len: 0x00020000 |
blk_size: 0x00004000 |initialized finish.
[D/FAL] (fal_flash_init:63) Flash device | onchip_flash | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 |initialized finish.
[D/FAL] (fal_flash_init:63) Flash device | nor_flash | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 |initialized finish.
[I/FAL] ===== FAL partition table =====
[I/FAL] | name      | flash_dev  | offset    | length    |
[I/FAL] -----
[I/FAL] | app        | onchip_flash | 0x00000000 | 0x000e0000 |
[I/FAL] | download   | nor_flash   | 0x00000000 | 0x00100000 |
[I/FAL] | factory    | nor_flash   | 0x00100000 | 0x00100000 |
[I/FAL] =====
[I/FAL] RT-Thread Flash Abstraction Layer (V0.4.0) initialize success.
[I/FAL] System initialization successful.
[I]RT-Thread OTA package(V0.2.1) initialize success.
[E]Get firmware header occur CRC32(calc.crc: 7b93c5c8 != hdr.info_crc32: ffffffff) error on 'download'
partition!
[E]Get OTA download partition firmware header failed!
[E]Get firmware header occur CRC32(calc.crc: 7b93c5c8 != hdr.info_crc32: ffffffff) error on 'app' partition!
[I]Begin to execute the program on app partition.
[I/FAL] Find user firmware at app partition 0x08020000 successfully.
[I/FAL] Bootloader jumps to user firmware now.

 \ | /
- RT -      Thread Operating System
/ | \      4.1.0 build Nov  9 2025 22:28:30
2006 - 2022 Copyright by RT-Thread team
[2] I/SFUD: Find a winbond flash chip. Size is 16777216 bytes.
[9] I/SFUD: norflash0 flash device is initialize success.
[16] I/SFUD: Probe SPI flash norflash0 by SPI device SPI1 success.
[D/FAL] (fal_flash_init:49) Flash device | onchip_flash_128k | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 |initialized finish.
[D/FAL] (fal_flash_init:49) Flash device | norflash0 | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 |initialized finish.
[D/FAL] (check_and_update_part_cache:140) warning: Do NOT found the flash device(stm32_onchip).
[I/FAL] ===== FAL partition table =====
[I/FAL] | name      | flash_dev  | offset    | length    |
[I/FAL] -----
[I/FAL] | app        | stm32_onchip | 0x00000000 | 0x000e0000 |
[I/FAL] | download   | norflash0   | 0x00000000 | 0x00100000 |
[I/FAL] | factory    | norflash0   | 0x00100000 | 0x00100000 |
[I/FAL] | env        | norflash0   | 0x00200000 | 0x00100000 |
[I/FAL] | fileysys   | norflash0   | 0x00300000 | 0x00400000 |
[I/FAL] | eflash     | norflash0   | 0x00700000 | 0x00900000 |
[I/FAL] =====
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[143] D/main: APP Version: 1.1.0 build Nov  9 2025

msh />
```

bootloader成功引导app程序。

7. 固件升级

由于BootLoader是基于fal操作的，即将download分区中存放的rbl文件解压后拷贝到指定分区。

使用RT-Thread的打包器时，有固件版本和固件分区名两个参数。

RT-Thread的BootLoader，在bootloader分区中存放app分区的**crc**、**时间戳**和**固件版本**，即当新下载的固件和原本固件内容相同时，不升级，避免无所谓的更新操作。

RT-Thread的BootLoader在升级的时候，对固件版本的判断是要求版本不一样（不需要新的版本号大于旧的，而且好像是通过字符串比对，即版本号你除了填写2.0.0，还可以填V2.0.0，或者Release2.0.0），如果软件重新编译了，时间戳是不一样的，对于不同时间戳但固件版本相同的两个固件，BootLoader也会执行升级操作的。

- 相同的时间戳，不同的固件版本，会升级
- 不同的时间戳，不同的固件版本，会升级
- 相同的时间戳，想同的固件版本，不会升级

BootLoader只能识别BootLoader中的分区表，app中的分区表无法识别。所以还是只能填APP

运行 packages\ota_downloader-v1.0.0\tools\ota_packager\rt_ota_packaging_tool.exe 固件打包器

- 打包 app 1.1.0 分区固件包：

```
main.c

#define APP_VERSION "1.1.0"
int main(void)
{
    ...
    LOG_D("APP version: %s build %s\r\n", APP_VERSION, __DATE__);
    ...
}
```

编译工程，生成bin文件。

打包 app 1.1.0 分区固件包：



RT-Thread OTA 固件打包器

选择固件

E:\YShengli\02-Notes\embs\07-gateway\06-rtthread-ota\

保存路径

E:\YShengli\02-Notes\embs\07-gateway\06-rtthread-ota\

压缩算法

不压缩

加密算法

不加密

加密密钥

加密 IV

固件分区名

app

固件版本

1.1.0

结果：**打包成功**

HASH_CODE F609BA59 RAW_SIZE : 151076
HDR_CRC32 : D28B7316 PKG_SIZE : 151076
BODY_CRC32: F847B7CA TIMESTAMP: 1762779628

开始打包

COPYRIGHT (C) 2012-2018, Shanghai Real-Thread Technology Co., Ltd Ver: 1.0.7

- 打包 app 1.2.0 分区固件包：

main.c

```
#define APP_VERSION "1.2.0"
int main(void)
{
    ...
    LOG_D("APP Version: %s build %s\r\n", APP_VERSION, __DATE__);
    ...
}
```

编译工程，生成bin文件。

打包 app 1.2.0 分区固件包：



RT-Thread OTA 固件打包器

选择固件

E:\YShengli\02-Notes\embs\07-gateway\06-rtthread-ota\

保存路径

E:\YShengli\02-Notes\embs\07-gateway\06-rtthread-ota\

压缩算法

不压缩

加密算法

不加密

加密密钥

加密 IV

固件分区名

app

固件版本

1.2.0

结果：**打包成功**

HASH_CODE F609BA59 RAW_SIZE : 151076
HDR_CRC32 : D28B7316 PKG_SIZE : 151076
BODY_CRC32: F847B7CA TIMESTAMP: 1762779628

开始打包

COPYRIGHT (C) 2012-2018, Shanghai Real-Thread Technology Co., Ltd Ver: 1.0.7

- 打包 app 1.3.0 分区固件包：

main.c

```
#define APP_VERSION "1.3.0"
int main(void)
{
    ...
    LOG_D("APP Version: %s build %s\r\n", APP_VERSION, __DATE__);
    ...
}
```

编译工程，生成bin文件。

打包 app 1.3.0 分区固件包：



RT-Thread OTA 固件打包器

选择固件

E:\YShengli\02-Notes\embs\07-gateway\06-rtthread-ota\

保存路径

E:\YShengli\02-Notes\embs\07-gateway\06-rtthread-ota\

压缩算法

不压缩

加密算法

不加密

加密密钥

加密 IV

固件分区名

app

固件版本

1.3.0

结果：**打包成功**

HASH_CODE F609BA59 RAW_SIZE : 151076
HDR_CRC32 : D28B7316 PKG_SIZE : 151076
BODY_CRC32: F847B7CA TIMESTAMP: 1762779628

开始打包

COPYRIGHT (C) 2012-2018, Shanghai Real-Thread Technology Co., Ltd Ver: 1.0.7

生成的固件如下：

07-gateway > 06-rtthread-ota > Debug >

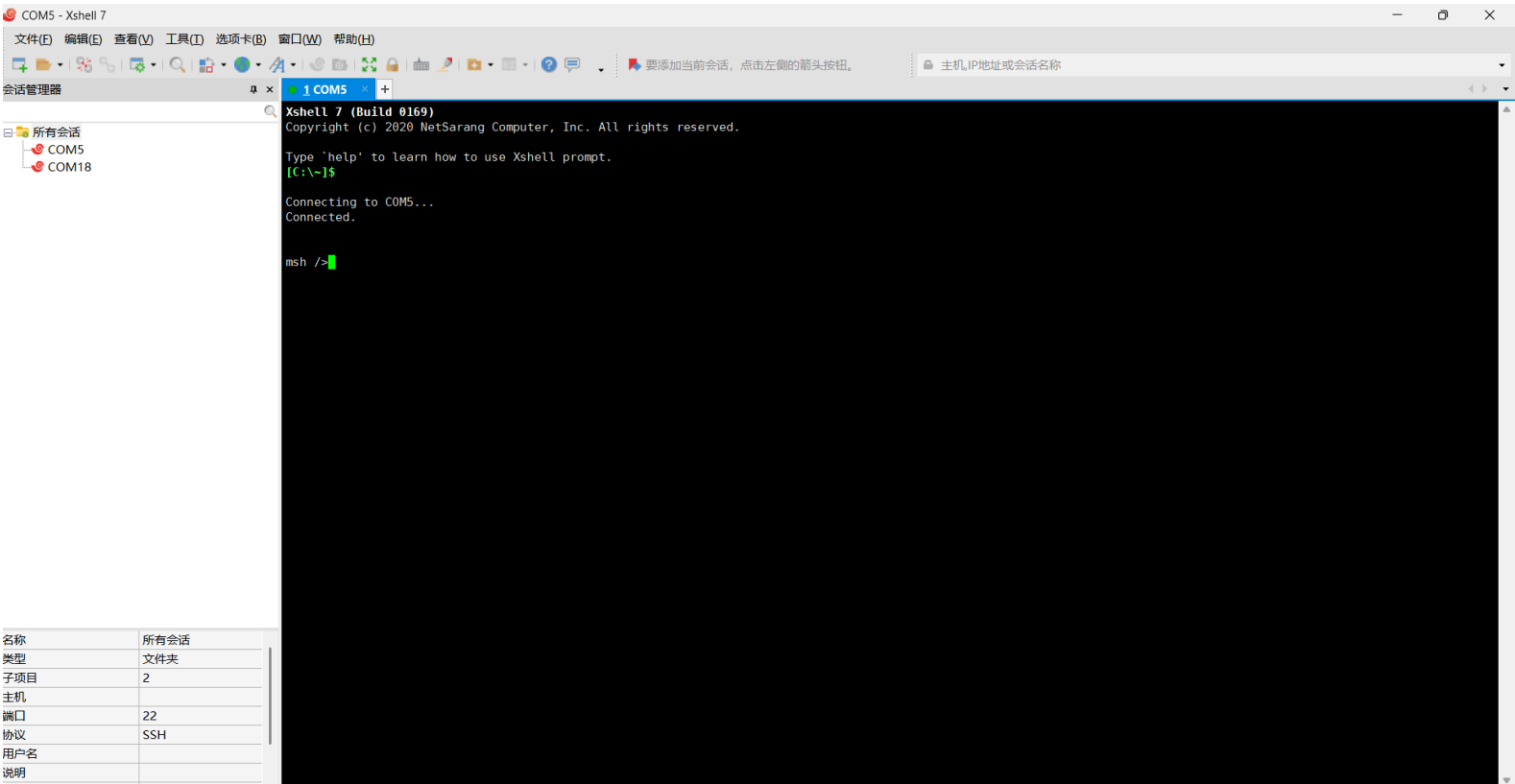
在 Debug 中搜索

名称	修改日期	类型	大小
applications	11/10 21:00	文件夹	
cubemx	11/9 21:36	文件夹	
drivers	11/10 19:04	文件夹	
libraries	11/9 21:33	文件夹	
packages	11/9 22:51	文件夹	
rt-thread	11/9 21:33	文件夹	
makefile	11/10 21:00	.	4 KB
objects.mk	11/10 19:04	MK 文件	1 KB
rtthread.bin	11/10 21:00	BIN 文件	148 KB
rtthread.elf	11/10 21:00	ELF 文件	800 KB
rtthread.map	11/10 21:00	Linker Address ...	667 KB
rtthread_app_1.1.0.rbl	11/10 20:55	RBL 文件	148 KB
rtthread_app_1.2.0.rbl	11/10 21:00	RBL 文件	148 KB
rtthread_app_1.3.0.rbl	11/10 20:51	RBL 文件	148 KB
sources.mk	11/10 21:00	MK 文件	2 KB

Ymodem下载固件

运行 STM32F407开发板V2\01-工具软件\Xshell-7.0\Xshell.exe , 配置并打开串口， 注意要先关闭 RT-Thread Studio 的终端，避免被占用。

实现 固件 1.1.0 -> 1.2.0 功能。



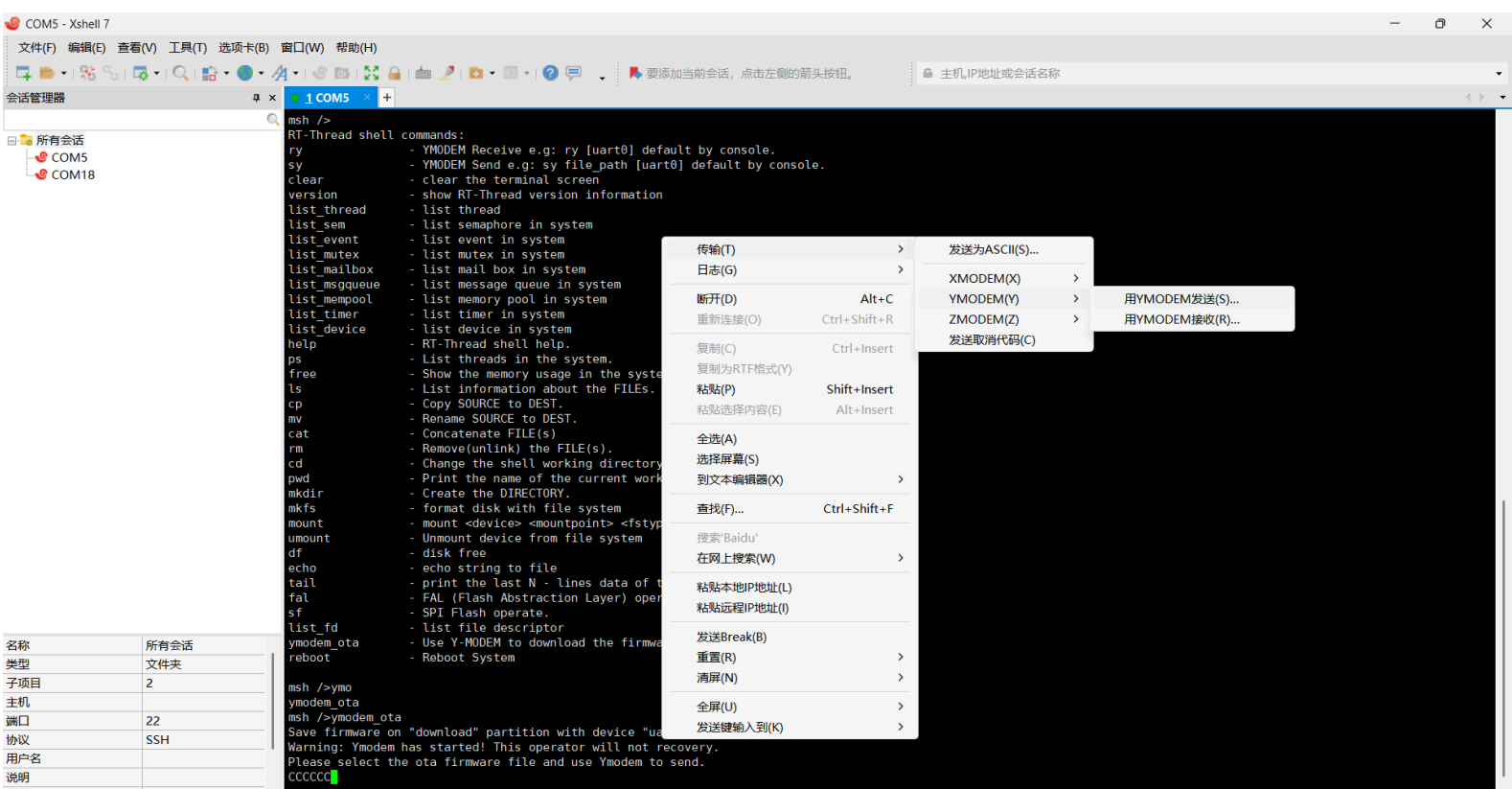
查看 ota 下载命令

```
msh />
RT-Thread shell commands:
ry                - YMODEM Receive e.g: ry [uart0] default by console.
sy                - YMODEM Send e.g: sy file_path [uart0] default by console.
clear             - clear the terminal screen
version           - show RT-Thread version information
list_thread       - list thread
list_sem          - list semaphore in system
list_event        - list event in system
list_mutex        - list mutex in system
list_mailbox      - list mail box in system
list_msgqueue     - list message queue in system
list_mempool      - list memory pool in system
list_timer        - list timer in system
list_device       - list device in system
help              - RT-Thread shell help.
ps                - List threads in the system.
free              - Show the memory usage in the system.
ls                - List information about the FILES.
cp                - Copy SOURCE to DEST.
mv                - Rename SOURCE to DEST.
cat               - Concatenate FILE(s)
rm                - Remove(unlink) the FILE(s).
cd                - Change the shell working directory.
pwd               - Print the name of the current working directory.
mkdir             - Create the DIRECTORY.
mkfs              - format disk with file system
mount             - mount <device> <mountpoint> <fstype>
umount            - Unmount device from file system
df                - disk free
echo              - echo string to file
tail              - print the last N - lines data of the given file
fal               - FAL (Flash Abstraction Layer) operate.
sf                - SPI Flash operate.
list_fd           - list file descriptor
ymodem_ota        - Use Y-MODEM to download the firmware
reboot            - Reboot System

msh />
```

下载固件

右击 -> 传输 -> YMODEM -> 用YMODEM发送。



选择要下载的文件，下载后，开始升级固件。

```
msh />ymodem_ota
Save firmware on "download" partition with device "uart1".
Warning: Ymodem has started! This operator will not recovery.
Please select the ota firmware file and use Ymodem to send.
CCCCDownload firmware to flash success.
System now will restart...
[SFUD]Find a winbond flash chip. Size is 16777216 bytes.
[SFUD]spi flash device is initialize success.

  _ _ _ _ _
|  ) |  _ |  ) / \ / \ |
| \ |  _ |  ) \ / \ /
2006 - 2019 Copyright by rt-thread team
0.9.2 build Nov 9 2025

[D/FAL] (fal_flash_init:63) Flash device | onchip_flash_16k_part | addr: 0x08000000 | len: 0x00020000 |
blk_size: 0x00004000 | initialized finish.
[D/FAL] (fal_flash_init:63) Flash device | onchip_flash | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 | initialized finish.
[D/FAL] (fal_flash_init:63) Flash device | nor_flash | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 | initialized finish.
[I/FAL] ===== FAL partition table =====
[I/FAL] | name | flash_dev | offset | length |
[I/FAL] |-----|
[I/FAL] | app | onchip_flash | 0x00000000 | 0x000e0000 |
[I/FAL] | download | nor_flash | 0x00000000 | 0x01000000 |
[I/FAL] | factory | nor_flash | 0x01000000 | 0x01000000 |
[I/FAL] |=====|
```

```
[I/FAL] RT-Thread Flash Abstraction Layer (V0.4.0) initialize success.
[I/FAL] System initialization successful.
[I]RT-Thread TA package(V0.2.1) initialize success.
[I]Verify 'download' partition(fw ver: 1.2.0, timestamp: 1762779628) success.
[I]OTA firmware(app) upgrade(1.1.0->1.2.0) startup.
[I]The partition 'app' is erasing.
[I]The partition 'app' erase success.
[I]OTA Write:
=====] 100%
[I]Verify 'app' partition(fw ver: 1.2.0, timestamp: 1762779628) success.
[I/FAL] Find user firmware at app partition 0x08020000 successfully.
[I/FAL] Bootloader jumps to user firmware now.

\ | /
- RT -      Thread Operating System
/ | \      4.1.0 build Nov 10 2025 19:04:46
2006 - 2022 Copyright by RT-Thread team
[2] I/SFUD: Find a winbond flash chip. Size is 16777216 bytes.
[9] I/SFUD: norflash0 flash device is initialize success.
[16] I/SFUD: Probe SPI flash norflash0 by SPI device SPI1 success.
[D/FAL] (fal_flash_init:49) Flash device |          onchip_flash | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 |initialized finish.
[D/FAL] (fal_flash_init:49) Flash device |          norflash0 | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 |initialized finish.
[I/FAL] ===== FAL partition table =====
[I/FAL] | name      | flash_dev  |  offset  |   length  |
[I/FAL] -----
[I/FAL] | app        | onchip_flash | 0x00020000 | 0x000e0000 |
[I/FAL] | download   | norflash0   | 0x00000000 | 0x00100000 |
[I/FAL] | factory    | norflash0   | 0x00100000 | 0x00100000 |
[I/FAL] | env        | norflash0   | 0x00200000 | 0x00100000 |
[I/FAL] | fileysys   | norflash0   | 0x00300000 | 0x00400000 |
[I/FAL] | eflash     | norflash0   | 0x00700000 | 0x00900000 |
[I/FAL] =====
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[133] D/main: APP Version: 1.2.0 build Nov 10 2025

msh />
```

• 升级固件1.2.0 -> 1.3.0

```
msh />ymodem_ota
Save firmware on "download" partition with device "uart1".
warning: Ymodem has started! This operator will not recovery.
Please select the ota firmware file and use Ymodem to send.
CCCDownload firmware to flash success.
System now will restart...
[SFUD]Find a winbond flash chip. Size is 16777216 bytes.
[SFUD]spi flash device is initialize success.

  _ _ _ _ _
|_) | _ |_) / \ / \ |
| \ |   |_) \_/ \_/ |
2006 - 2019 Copyright by rt-thread team
          0.9.2 build Nov 9 2025
[D/FAL] (fal_flash_init:63) Flash device |          onchip_flash_16k_part | addr: 0x08000000 | len: 0x00020000 |
blk_size: 0x00004000 |initialized finish.
[D/FAL] (fal_flash_init:63) Flash device |          onchip_flash | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 |initialized finish.
[D/FAL] (fal_flash_init:63) Flash device |          nor_flash | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 |initialized finish.
[I/FAL] ===== FAL partition table =====
[I/FAL] | name      | flash_dev  |  offset  |   length  |
[I/FAL] -----
[I/FAL] | app        | onchip_flash | 0x00000000 | 0x000e0000 |
[I/FAL] | download   | nor_flash   | 0x00000000 | 0x00100000 |
[I/FAL] | factory    | nor_flash   | 0x00100000 | 0x00100000 |
[I/FAL] =====
[I/FAL] RT-Thread Flash Abstraction Layer (V0.4.0) initialize success.
[I/FAL] System initialization successful.
[I]RT-Thread TA package(V0.2.1) initialize success.

[I]Verify 'download' partition(fw ver: 1.3.0, timestamp: 1762779084) success.
[I]OTA firmware(app) upgrade(1.2.0->1.3.0) startup.
[I]The partition 'app' is erasing.
[I]The partition 'app' erase success.
[I]OTA Write:
=====] 100%
[I]Verify 'app' partition(fw ver: 1.3.0, timestamp: 1762779084) success.
[I/FAL] Find user firmware at app partition 0x08020000 successfully.
[I/FAL] Bootloader jumps to user firmware now.

\ | /
- RT -      Thread Operating System
/ | \      4.1.0 build Nov 10 2025 19:04:46
2006 - 2022 Copyright by RT-Thread team
[2] I/SFUD: Find a winbond flash chip. Size is 16777216 bytes.
[9] I/SFUD: norflash0 flash device is initialize success.
[16] I/SFUD: Probe SPI flash norflash0 by SPI device SPI1 success.
[D/FAL] (fal_flash_init:49) Flash device |          onchip_flash | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 |initialized finish.
[D/FAL] (fal_flash_init:49) Flash device |          norflash0 | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 |initialized finish.
[I/FAL] ===== FAL partition table =====
[I/FAL] | name      | flash_dev  |  offset  |   length  |
[I/FAL] -----
[I/FAL] | app        | onchip_flash | 0x00020000 | 0x000e0000 |
[I/FAL] | download   | nor_flash   | 0x00000000 | 0x00100000 |
[I/FAL] | factory    | nor_flash   | 0x00100000 | 0x00100000 |
[I/FAL] | env        | norflash0   | 0x00200000 | 0x00100000 |
[I/FAL] | fileysys   | norflash0   | 0x00300000 | 0x00400000 |
[I/FAL] | eflash     | norflash0   | 0x00700000 | 0x00900000 |
[I/FAL] =====
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[133] D/main: APP Version: 1.3.0 build Nov 10 2025

msh />
```

• 升级固件1.3.0 -> 1.1.0

```
msh />ymodem_ota
Save firmware on "download" partition with device "uart1".
warning: Ymodem has started! This operator will not recovery.
Please select the ota firmware file and use Ymodem to send.
CCCDownload firmware to flash success.
System now will restart...
[SFUD]Find a winbond flash chip. Size is 16777216 bytes.
[SFUD]spi flash device is initialize success.

  _ _ _ _ _
|_) | _ |_) / \ / \ |
| \ |   |_) \_/ \_/ |
2006 - 2019 Copyright by rt-thread team
          0.9.2 build Nov 9 2025
[D/FAL] (fal_flash_init:63) Flash device |          onchip_flash_16k_part | addr: 0x08000000 | len: 0x00020000 |
blk_size: 0x00004000 |initialized finish.
[D/FAL] (fal_flash_init:63) Flash device |          onchip_flash | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 |initialized finish.
[D/FAL] (fal_flash_init:63) Flash device |          nor_flash | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 |initialized finish.
[I/FAL] ===== FAL partition table =====
[I/FAL] | name      | flash_dev  |  offset  |   length  |
[I/FAL] -----
[I/FAL] | app        | onchip_flash | 0x00000000 | 0x000e0000 |
[I/FAL] | download   | nor_flash   | 0x00000000 | 0x00100000 |
[I/FAL] | factory    | nor_flash   | 0x00100000 | 0x00100000 |
[I/FAL] =====
[I/FAL] RT-Thread Flash Abstraction Layer (V0.4.0) initialize success.
[I/FAL] System initialization successful.
[I]RT-Thread TA package(V0.2.1) initialize success.
[I]Verify 'download' partition(fw ver: 1.1.0, timestamp: 1762779142) success.
[I]OTA firmware(app) upgrade(1.3.0->1.1.0) startup.
[I]The partition 'app' is erasing.
[I]The partition 'app' erase success.
[I]OTA Write:
=====] 100%
[I]Verify 'app' partition(fw ver: 1.1.0, timestamp: 1762779142) success.
[I/FAL] Find user firmware at app partition 0x08020000 successfully.
[I/FAL] Bootloader jumps to user firmware now.

\ | /
- RT -      Thread Operating System
/ | \      4.1.0 build Nov 10 2025 19:04:46
2006 - 2022 Copyright by RT-Thread team
[2] I/SFUD: Find a winbond flash chip. Size is 16777216 bytes.
[9] I/SFUD: norflash0 flash device is initialize success.
```

```
[16] I/SFUD: Probe SPI flash norflash0 by SPI device SPI1 success.
[D/FAL] (fal_flash_init:49) Flash device | onchip_flash | addr: 0x08020000 | len: 0x000e0000 |
blk_size: 0x00020000 |initialized finish.
[D/FAL] (fal_flash_init:49) Flash device | norflash0 | addr: 0x00000000 | len: 0x01000000 |
blk_size: 0x00001000 |initialized finish.
[I/FAL] ===== FAL partition table =====
[I/FAL] | name | flash_dev | offset | length |
[I/FAL] -----
[I/FAL] | app | onchip_flash | 0x00020000 | 0x000e0000 |
[I/FAL] | download | norflash0 | 0x00000000 | 0x00100000 |
[I/FAL] | factory | norflash0 | 0x00100000 | 0x00100000 |
[I/FAL] | env | norflash0 | 0x00200000 | 0x00100000 |
[I/FAL] | filesys | norflash0 | 0x00300000 | 0x00400000 |
[I/FAL] | eflash | norflash0 | 0x00700000 | 0x00900000 |
[I/FAL] =====
[I/FAL] RT-Thread Flash Abstraction Layer initialize success.
[133] D/main: APP Version: 1.1.0 build Nov 10 2025

msh />
```

8. factory恢复固件

按下ESC按键不松手， 再按下复位键， 即可实现恢复出厂设置功能。

2.3 练习 07 RT-Thread 环境变量

1. 题目要求

- 设置和保存环境变量到片外flash内。
- 用于保存mqtt服务器的token。

2. 配置 CubeMX

3. OTA固件测试地址

http_ota http://static.getiot.tech/rt-thread.rbl

http_tb_ota xthings.cloud 80 2xGnIJWwyKeMI3hMvpOt

2.4 练习 08 RT-Thread http 升级固件

1. 题目要求

- 设置和保存环境变量到片外flash内。
- 用于保存mqtt服务器的token。

2. 编译报错

- 错误1

```
../packages/webclient-v2.2.0/src/webclient.c: In function 'webclient_send':
../packages/webclient-v2.2.0/src/webclient.c:93:12: warning: passing argument 6 of 'sal_sendto' makes
integer from pointer without a cast [-Wint-conversion]
    return send(session->socket, buffer, len, flag);
           ^
In file included from E:\YShengli\02-Notes\emb2507\07-gateway\08-rtthread-ota-http\rt-
thread\components\net\sai\include\sai_netdb.h:14:0,
               from E:\YShengli\02-Notes\emb2507\07-gateway\08-rtthread-ota-http\rt-
thread\components\net\sai\include\socket\netdb.h:20,
               from ../packages/webclient-v2.2.0/src/webclient.c:30:
E:\YShengli\02-Notes\emb2507\07-gateway\08-rtthread-ota-http\rt-
thread\components\net\sai\include\sai_socket.h:208:5: note: expected 'socklen_t {aka long unsigned int}' but
argument is of type 'void *'
    int sal_sendto(int socket, const void *dataptr, size_t size, int flags,
```

修改send函数:

rt-thread\components\net\sai\include\socket\sys_socket\sys\socket.h

```
#define send(s, dataptr, size, flags) sal_sendto(s, dataptr, size, flags, NULL,
(int)NULL)
```

- 警告2:

```
E:\YShengli\02-Notes\emb2507\07-gateway\08-rtthread-ota-http\rt-
thread\components\net\sai\include\sai_socket.h:208:5: note: expected 'socklen_t {aka long unsigned int}' but
argument is of type 'void *'
    int sal_sendto(int socket, const void *dataptr, size_t size, int flags,
           ^
../packages/webclient-v2.2.0/src/webclient.c: In function 'webclient_shard_position_function':
../packages/webclient-v2.2.0/src/webclient.c:1151:66: warning: pointer targets in passing argument 3 of
'webclient_response' differ in signedness [-Wpointer-sign]
    data_len = webclient_response(session, (void **)&buffer, &resp_len);
                                           ^
In file included from ../packages/webclient-v2.2.0/src/webclient.c:23:0:
E:\YShengli\02-Notes\emb2507\07-gateway\08-rtthread-ota-http\packages\webclient-
v2.2.0\inc\webclient.h:138:5: note: expected 'size_t * {aka unsigned int *}' but argument is of type 'int *'
    int webclient_response(struct webclient_session *session, void **response, size_t *resp_len);
```

- 修改为:

```
size_t resp_len = 0;
```

- 警告3

```
../packages/ota_downloader-v1.0.0/src/http_ota.c: In function 'http_ota_shard_download_handle':
../packages/ota_downloader-v1.0.0/src/http_ota.c:82:52: warning: pointer targets in passing argument 3 of
'fal_partition_write' differ in signedness [-Wpointer-sign]
    if (fal_partition_write(dl_part, begin_offset, buffer, length) < 0)
           ^
In file included from ../packages/ota_downloader-v1.0.0/src/http_ota.c:17:0:
E:\YShengli\02-Notes\emb2507\07-gateway\08-rtthread-ota-http\rt-thread\components\fal\inc\fal.h:90:5: note:
expected 'const uint8_t * {aka const unsigned char *}' but argument is of type 'char *'
    int fal_partition_write(const struct fal_partition *part, uint32_t addr, const uint8_t *buf, size_t size);
```

- 修改为:

```
if (fal_partition_write(dl_part, begin_offset, (uint8_t *)buffer, length) < 0)
```

3. 下载固件

http_tb_ota xthings.cloud 80 2xGnIJWwyKeMI3hMvpOt

设置环境变量

packages\EasyFlash-v4.1.0\src\ef_fal_port.c

```
/* default ENV set for user */
static const ef_env default_env_set[] = {
    {"tb_ota_host", "xthings.cloud"},
    {"tb_ota_port", "80"},
    {"tb_ota_token", "123456789"},
    {"tb_ota_update", "false"},
};
```


源程序

main.c

```
/*
 * Copyright (c) 2006-2025, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2025-11-12     RT-Thread     first version
 */

#include <rtthread.h>

#define DBG_TAG "main"
#define DBG_LVL DBG_LOG
#include <rtdbg.h>

#define APP_VERSION "1.1.0"

int main(void)
{
    int count = 1;
    extern int http_tb_ota_init(void);
    http_tb_ota_init();
    LOG_D("APP Version: %s build %s\r\n", APP_VERSION, __DATE__);
    while (count++)
    {
        rt_thread_mdelay(1000);
    }

    return RT_EOK;
}
```